

Dynamic Programming & Reinforcement Learning

Instructor: Chuanhao Li

Sep 15, 2026

Content for Today



- **Course Overview**

- Logistics and administration
- What we will learn

- **Introduction**

- Sequential decision-making problems
- Dynamic programming vs reinforcement learning
- More on reinforcement learning

Course Overview



About the Team



- Instructor: Chuanhao Li (李传浩)
 - Assistant Professor in Department of IE
 - Office: Shunde Building. (舜德楼), Room 402B
 - Email: chuanhao-li@tsinghua.edu.cn
- Instructor: Chengli Zhu (朱成礼)
 - Senior Engineer in Department of IE
 - Email: zhcl@tsinghua.edu.cn
 - Responsible for the two lab sessions that we will have in this semester
- TA: Zhengyu Ye (叶征宇)
 - 2nd year PhD Student in Department of IE
 - Email: zy-ye20@mails.tsinghua.edu.cn
 - Responsible for assignment-related questions and grading

About the Course



- **Sequential decision-making problems**
 - Formulations
 - Applications
- **How to solve sequential decision-making problems**
 - Planning/dynamic programming (when you know how the environment changes)
 - Reinforcement learning (when you don't know that and needs to estimate)
- **Technical details**
 - Bellman equations, value iteration, policy iteration
 - Online planning/search methods
 - Exploration vs exploitation, regret minimization
 - Function approximations, deep RL, policy gradients
 - Adaptation to multi-agents, equilibrium computing methods + RL

Tentative Course Topics



清华大学
Tsinghua University



Course objective

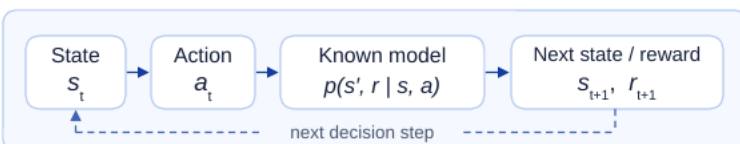
Master dynamic programming and reinforcement learning for modeling, analyzing and solving sequential decision problems.

1 Dynamic Programming (DP)

Planning with an explicit model

Use a transition and reward model to evaluate and improve sequential decisions.

MODEL-BASED DECISION STEP



Main topics



Modeling

- States, actions and objectives
- Deterministic and stochastic models
- Finite and infinite horizons



Core methods

- Bellman equations and optimality
- Backward induction
- Value iteration
- Policy iteration



Scaling up

- Approximate DP
- Value approximation and aggregation
- Rollout and lookahead

Key idea: Use Bellman recursion to evaluate values and improve decisions across stages.

Shared foundation

Optimize expected cumulative reward or cost through sequential decisions.

States · actions · rewards
Policies · value functions

Markov decision processes (MDPs)



Planning

Evaluate and improve policies using a model.

Learning

Estimate values, policies or dynamics from experience.

Complementary approaches

Model-based RL

Learn a model, then plan.

Approximate DP

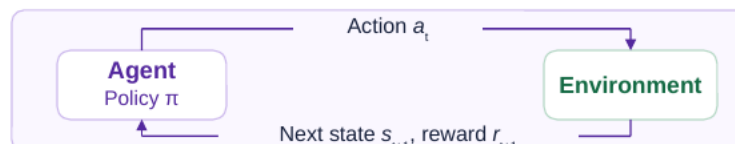
Use approximation to scale planning.

2 Reinforcement Learning (RL)

Learning from interaction or logged experience

Learn decision policies from interaction with the environment or from previously collected experience.

AGENT-ENVIRONMENT INTERACTION



Main topics



Exploration

- Multi-armed bandits
- ϵ -greedy and optimism
- Thompson sampling



Learning & control

- Monte Carlo and TD learning
- SARSA and Q-learning
- On-policy vs. off-policy



Function approximation

- Value and policy networks
- Policy gradient methods
- Actor-critic methods



Selected extensions

- POMDPs and belief states
- Constraints and risk sensitivity
- Offline and transfer RL

Key idea: Use experience to estimate values or policies and improve expected long-term performance.

3

Multi-Agent Reinforcement Learning (MARL)

Foundations Learning and decision making with multiple agents in a shared environment.



Cooperation & competition



Game-theoretic foundations



Centralized training;
decentralized execution



Multi-agent learning methods

Representative applications



Production & inventory management



Logistics & transportation



Finance & investment



Robotics & control



Recommendation & personalization



Energy & resource management

Assessment



- Attendance and participation
- Homework
 - Derivation of some fundamental theorems and programming tasks
- In-class quizzes
 - Mainly based on class material and homework
- Two lab sessions
 - In the middle and at the end of the semester

Announcements & Homework Submissions: <https://learn.tsinghua.edu.cn> (清华大学网络学堂) → 动态规划与强化学习(40160923-0)



- Allowed for homework but not for in-class quizzes.
- Goal
 - The important thing is to learn the knowledge, not completing the tasks.
 - In the era of AI, we need to be “verifiers”, not “routers” of LLMs.

Background and preparation



Background	Where it will be used
Probability	Conditional probability, expectation, sampled estimates
Linear algebra and calculus	Value equations, approximation, gradients
Python Programming	Implementing DP and RL algorithms



Dynamic Programming and Optimal Control, Volume I

Dimitri P. Bertsekas

Modeling, the principle of optimality, and classical DP

Reinforcement Learning: An Introduction, 2nd edition

Richard S. Sutton and Andrew G. Barto

Learning from experience, value functions, and policy optimization

Reinforcement Learning: Theory and Algorithms

Alekh Agarwal Kianté Brantley Nan Jiang Sham M. Kakade Wen Sun

Bandits, exploration, regret analysis

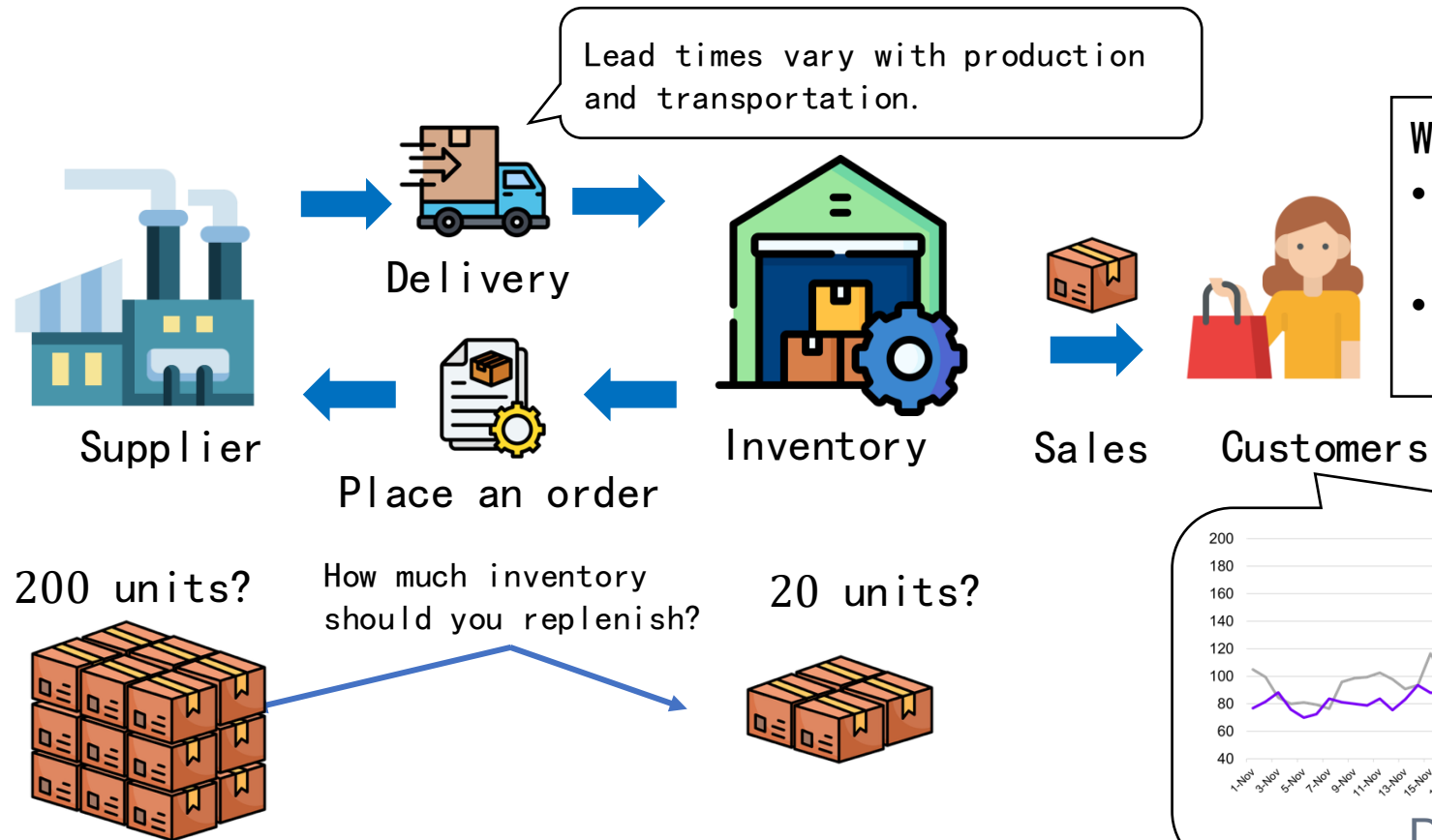
Example Sequential Decision-Making Problems



Sequential Decisions in Inventory Management

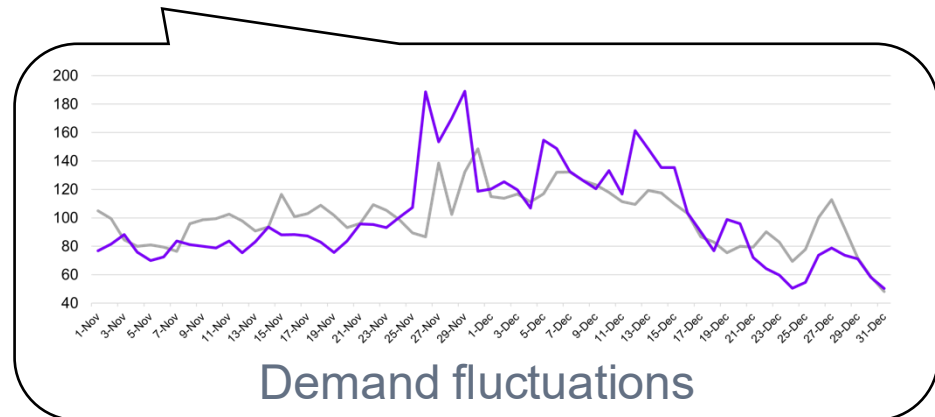


- You manage inventory for a product sold on an e-commerce platform.



Why is this sequential?

- Observe inventory and demand, then choose how much to order.
- Each order changes future inventory and the decisions available later.



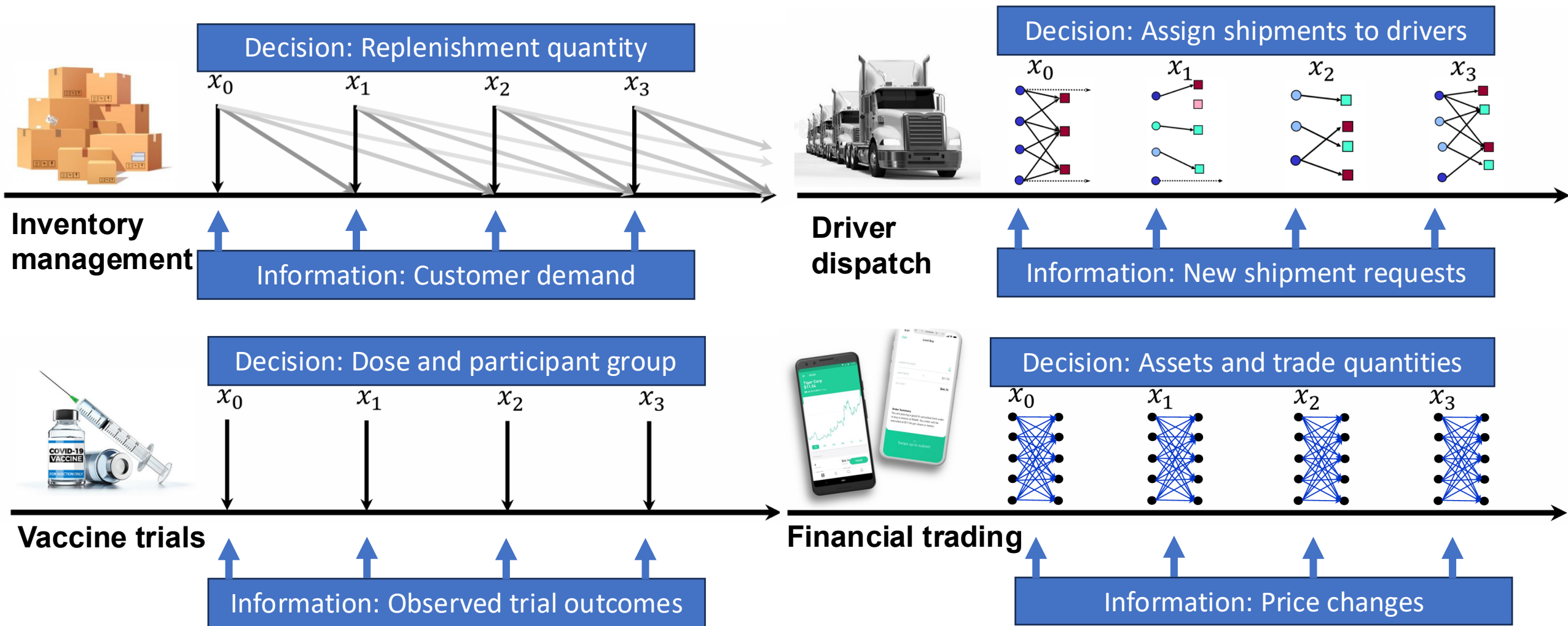
- More safety stock;
- Higher holding cost.
- Lower holding cost;
- stockout risk and lost sales.

More Examples



清华大学
Tsinghua University

- Decision $\rightarrow$ new information $\rightarrow$ decision $\rightarrow$ new information $\rightarrow$...



Markov Decision-making Process

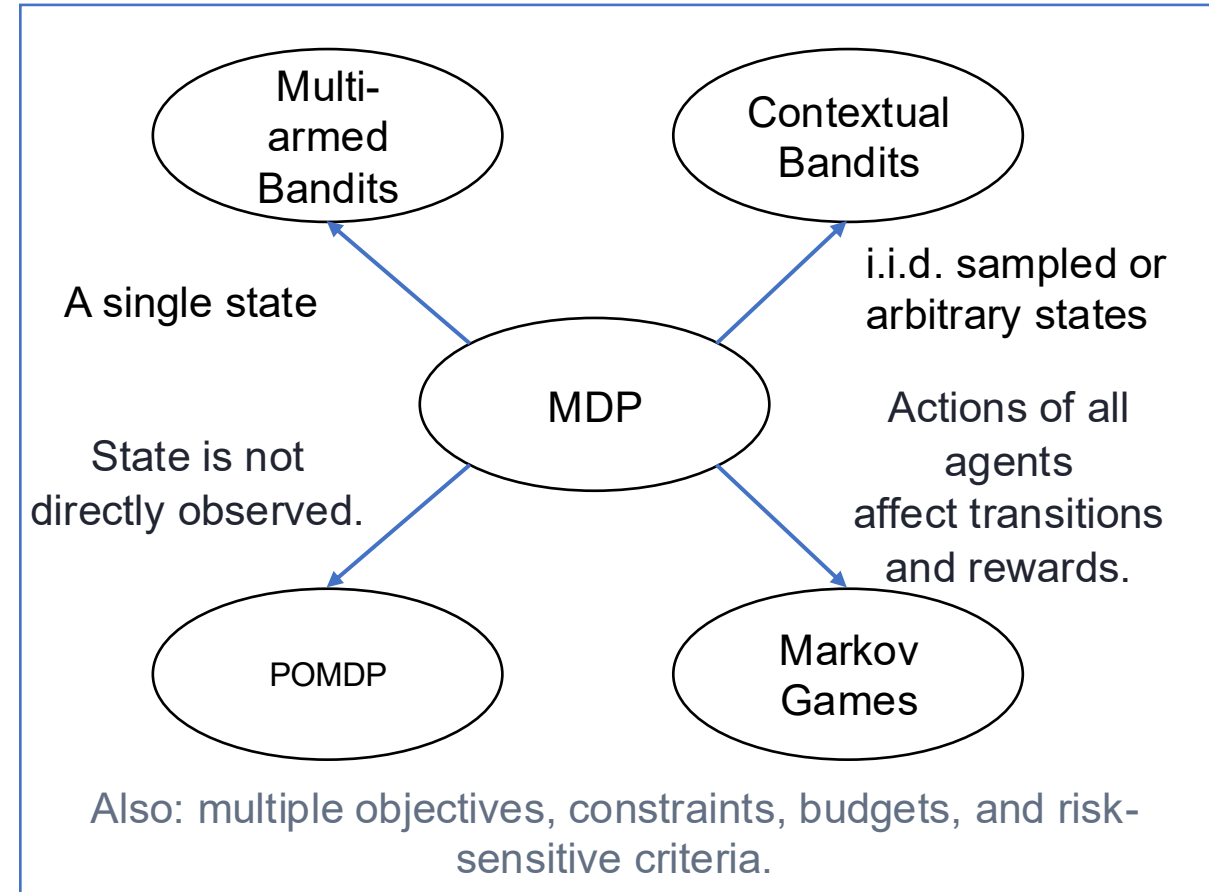


- **Core components**

- $\mathcal{A}$: Action space
- $\mathcal{S}$: State space
- $p: \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$: State transition function
- $r: \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$: Reward function

- Why we use this formulation

- A common modeling language, with special cases and extensions for many settings.





EXAMPLE 1

Resource Allocation

Deterministic dynamic programming

Adding uncertainty in resource use and profit



Problem Description



- Three products require different resources and generate different profits.

Product	Resource requirement (units per item)	Profit (RMB thousands per item)
1	$h_1 = 2$	$r_1 = 31$
2	$h_2 = 3$	$r_2 = 47$
3	$h_3 = 1$	$r_3 = 14$

- **Available resources: 4 units in total.**
- Goal: Choose production quantities to maximize total profit.

Recall: Mixed Integer Linear Programming (MILP)



- Decision variables

A_1, A_2, A_3 : Number of units to produce of products 1, 2, 3

- Objective: maximize total profit

$$\max_{A_1, A_2, A_3} 31A_1 + 47A_2 + 14A_3$$

- Constraints

- 1. resource capacity: $2A_1 + 3A_2 + A_3 \leq 4$
- 2. All variables are nonnegative integers: $A_1, A_2, A_3 \in \mathbb{Z}_+$ (非负整数)

An Alternative Method: Dynamic Programming



- $T = 3$ stages: decide quantity of products $1 \rightarrow 2 \rightarrow 3$
- Action $A_t \in \{0, 1, \dots, \lfloor 4/h_t \rfloor\}$: quantity of product $t \in [T]$ to produce
- State $S_t \in \{0, 1, \dots, 4\}$: resource available at stage t , initial state $S_1 = (4)$
- State transition function
 - Remaining resource after producing product t : $S_{t+1} = (S_t - h_t A_t)$
- Update rule: 按 $t = 3, 2, 1$ 的顺序逆向计算 (设 $V_4(S) = 0$)

$$V_t(S_t) = \max_{A_t} \{R(S_t, A_t) + V_{t+1}(S_{t+1})\}$$



生产 A_t 个产品 t 的利润
(即时奖励)



剩余资源能带来的最大利润
(状态价值函数)

Do Both Methods Give the Same Answer?



Yes. For this deterministic example, both methods optimize the same objective over the same feasible production plans.

Same objective

Maximize total profit under the same resource budget and integer production requirements.

Equivalent feasibility

MILP imposes a linear budget constraint.

DP enforces that budget through feasible actions and state transitions.

Global optimization

Solving the MILP to optimality and applying exact backward induction yield the same optimal value.

DP compares each action together with its optimal continuation—not just its immediate profit.

Backward Induction: Quantity of Product 3



(每个需要1单位资源, 利润14千元)

动作: 产品3数量

$$V_3(S_3) = \max_{A_3} \{R(S_3, A_3)\}$$

各状态下最优收益 V_3

状态: 剩余资源	0	1	2	3	4
4					
3					/
2				/	/
1			/	/	/
0		/	/	/	/

Backward Induction: Quantity of Product 3



(每个需要1单位资源, 利润14千元)

动作: 产品3数量

$$V_3(S_3) = \max_{A_3} \{R(S_3, A_3)\}$$

各状态下最优收益 V_3

		0	1	2	3	4	各状态下最优收益 V_3
状态: 剩余资源	4	0	14	2×14	3×14	4×14	
	3	0	14	2×14	3×14	/	
	2	0	14	2×14	/	/	
	1	0	14	/	/	/	
	0	0	/	/	/	/	
	0	0	/	/	/	/	0

Backward Induction: Quantity of Product 2



(每个需要3单位资源，利润47千元)

动作：产品2数量

状态：
剩余资源

	0	1
4		
3		
2		/
1		/
0		/

$$V_2(S_2) = \max_{A_2} \{R(S_2, A_2) + V_3(S_3)\}$$

各状态下最优收益 V_2

Backward Induction: Quantity of Product 2



(每个需要3单位资源, 利润47千元)

动作: 产品2数量

		0	1
状态: 剩余资源	4	0 + $V_3(4)$	47 + $V_3(1)$
	3	0 + $V_3(3)$	47 + $V_3(0)$
	2	0 + $V_3(2)$	/
	1	0 + $V_3(1)$	/
	0	0	/

$$V_2(S_2) = \max_{A_2} \{R(S_2, A_2) + V_3(S_3)\}$$

各状态下最优收益 V_2

61
47
28
14
0

Backward Induction: Quantity of Product 1



(每个需要2单位资源, 利润31千元)

动作: 产品1数量

状态:
剩余资源

	0	1	2
4			
3			/
2			/
1			/
0		/	/

$$V_1(S_1) = \max_{A_1} \{R(S_1, A_1) + V_2(S_2)\}$$

各状态下最优收益 V_1

Backward Induction: Quantity of Product 1



(每个需要2单位资源, 利润31千元)

动作: 产品1数量

		0	1	2
状态: 剩余资源	4	0 + $V_2(4)$	1×31 + $V_2(2)$	2×31 + $V_2(0)$
	3	0 + $V_2(3)$	1×31 + $V_2(1)$	/
	2	0 + $V_2(2)$	1×31 + $V_2(0)$	/
	1	0 + $V_2(1)$	/	/
	0	0	/	/

$$V_1(S_1) = \max_{A_1} \{R(S_1, A_1) + V_2(S_2)\}$$

各状态下最优收益 V_1

62
45
31
14
0

结论: 生产

$$A_1 = 2,$$

$$A_2 = 0,$$

$$A_3 = 0,$$

的利润最大。

引入资源消耗与利润的随机性

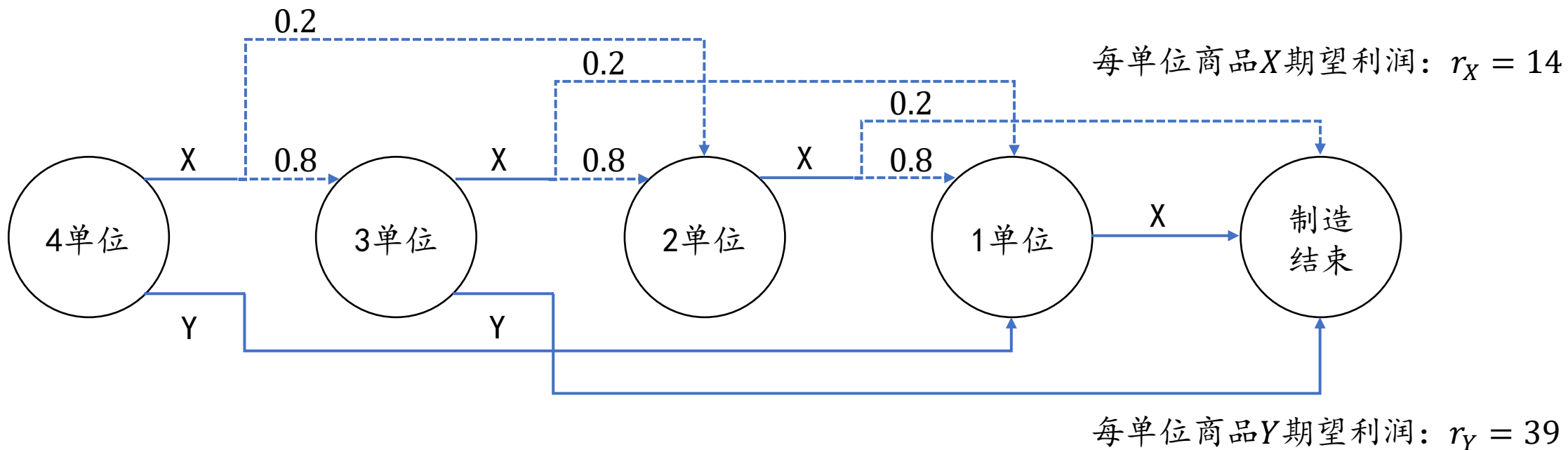


- 分配4单位生产资源，最大化期望利润
- 产品X
 - 资源消耗:
 - 0.8的概率消耗1单位资源
 - 0.2的概率消耗2单位资源

} 选择产品X后的状态是随机的
 - 利润：每个产品X的利润固定为14千元 期望利润 $r_X = 14$
- 产品Y
 - 资源消耗：固定消耗3单位资源
 - 利润：
 - 0.5的概率获得31千元利润
 - 0.5的概率获得47千元利润

} 期望利润 $r_Y = 0.5 \times 31 + 0.5 \times 47 = 39$

Illustration of State Transition



与示例1的区别

- 随机性：一个产品完成前，资源消耗是不确定的，即下一状态不确定
- 动态决策需求：不同于之前固定三次决策，当前问题中剩余资源量决定了可行的下一步选择

Formulate as MDP



- 资源耗尽时结束决策，因此总次数 T 不固定，但满足 $T \leq 4$
- 动作 $A_t \in \{X, Y\}$: 生产产品的类别
- 状态 $S_t \in \{0, 1, \dots, 4\}$: 第 t 次决策时的可用资源，初始状态 $S_1 = (4)$
- 状态转移函数
 - 生产某个产品 A_t 后，系统进入新状态：

$$\text{当 } A_t = X \text{ 时, } S_{t+1} = \begin{cases} S_t - 1, & 80\% \text{ 的概率} \\ S_t - 2, & 20\% \text{ 的概率} \end{cases}$$

$$\text{当 } A_t = Y \text{ 时, } S_{t+1} = S_t - 3$$

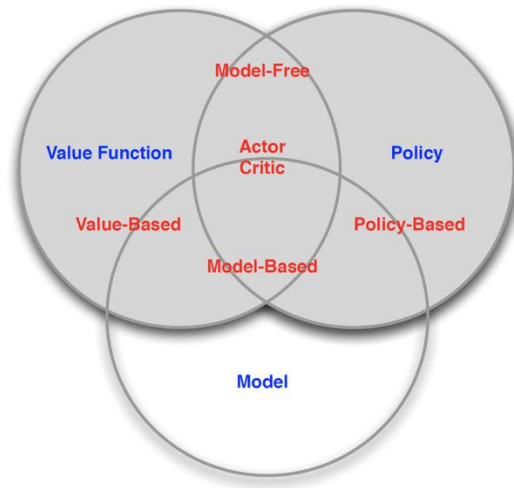


如果状态转移函数已知，可类似地通过动态规划求最优策略



如果状态转移函数是未知的该怎么办？

Reinforcement Learning



*Figure credit: David Silver,
"Introduction to RL"*



Reinforcement learning is learning what to do — how to map situations to actions — so as to maximize a numerical reward signal.
- Sutton & Barto, 2018

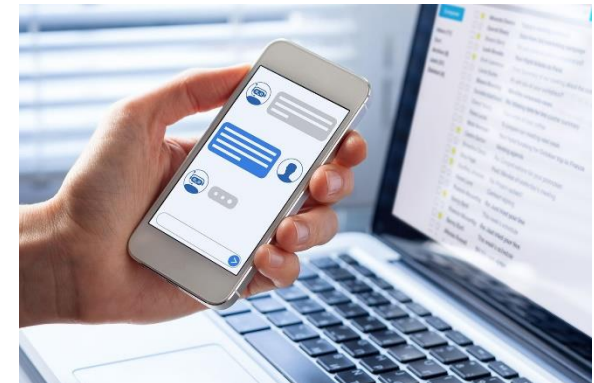
Reinforcement learning (RL) is an area of machine learning concerned with how software agents ought to take actions in an environment in order to maximize the notion of cumulative reward.
- Wikipedia



Game playing



Self-driving car



Conversational System

➤ A typical (narrow) view of the problem formulation

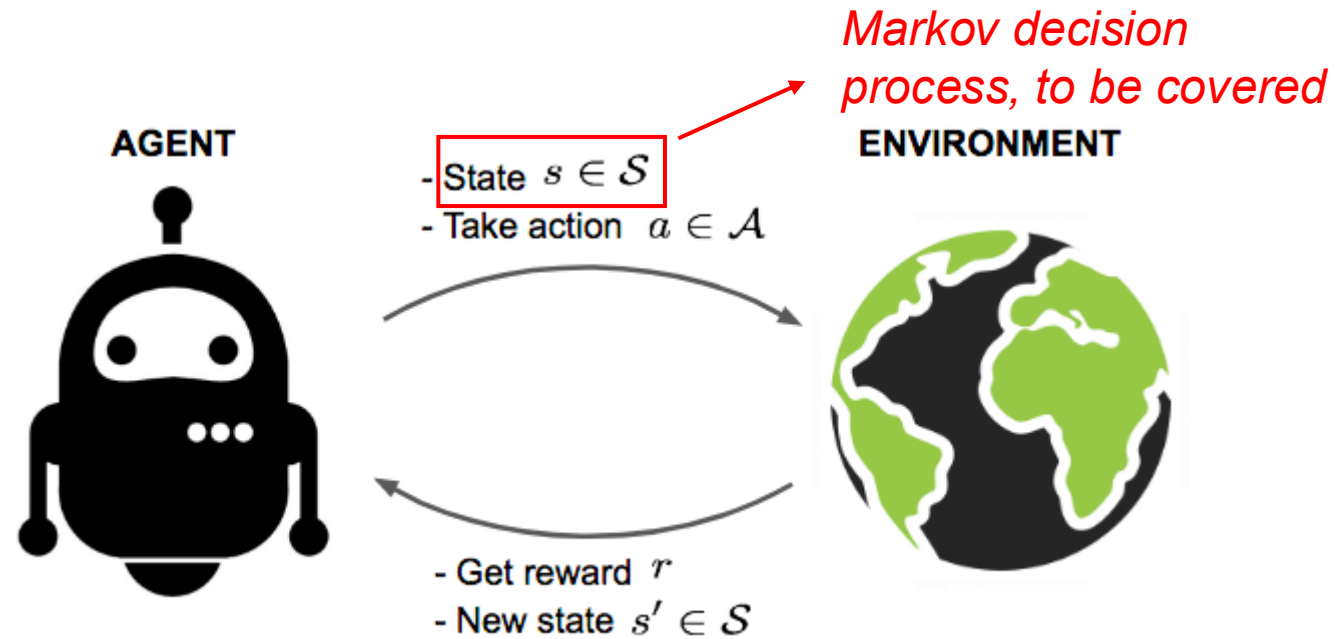
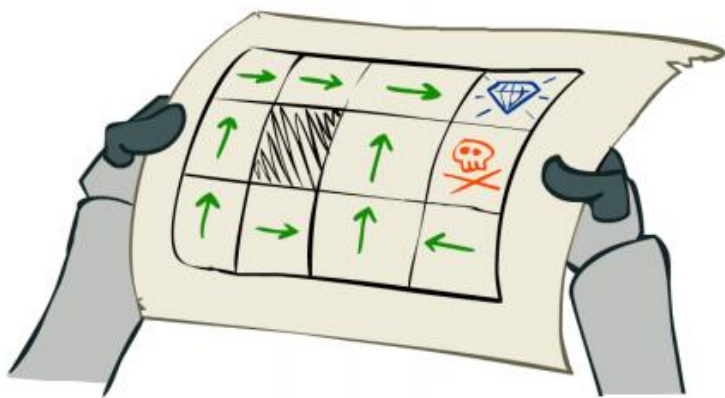


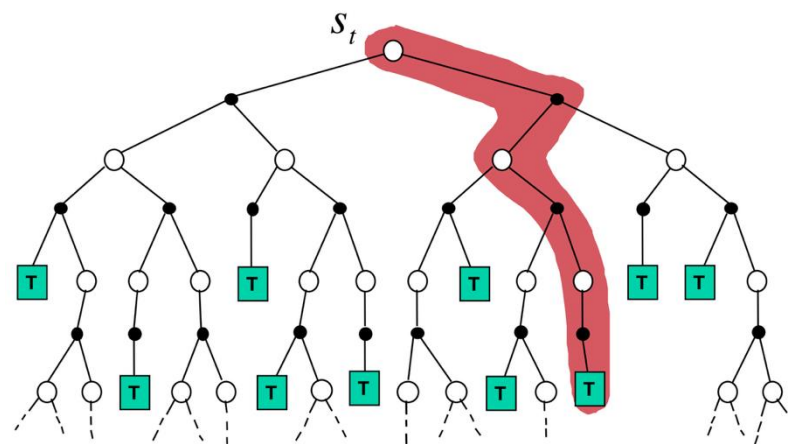
Image credit: Lil'Log

➤ It is a family of solutions

- Taking a series of actions to maximum cumulative return



Planning



~~Planning while learning~~
Reinforcement

Image credit: David Silver,
"Model-Free Prediction"

➤ It is a collection of fields that study such solutions

- ❑ Computer science, psychology, neuroscience, optimization, operations research, and many others

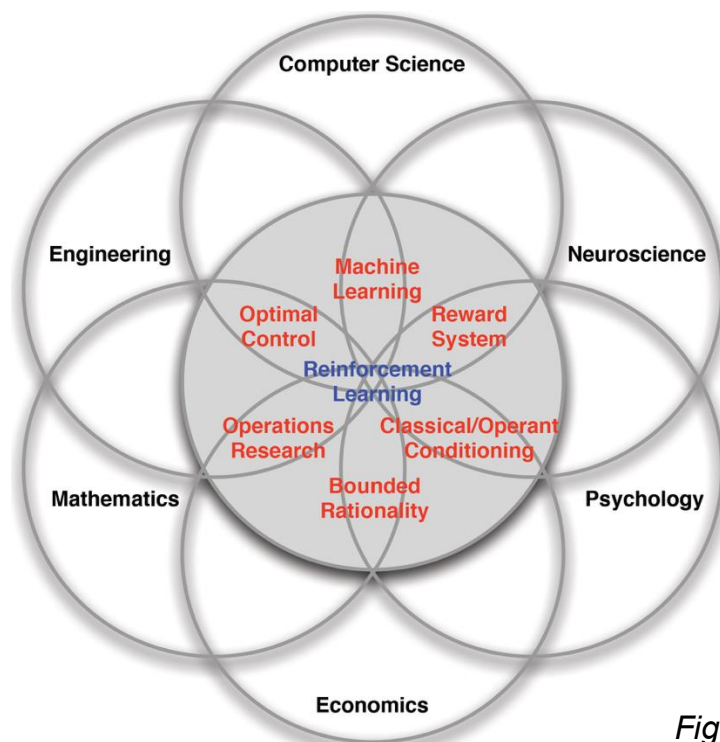


Figure credit: David Silver,
"Introduction to RL"

Vs. Supervised machine learning

➤ Classification as an example

□ Training time

- Input: $\{(\mathbf{x}_n, y_n)\}_{n=1}^N$, where $\mathbf{x}_n \in \mathbb{R}^d$ and $y_n \in [C]$
- Output: hypothesis $f_\theta(\mathbf{x}) \rightarrow y$
- Goal: $\min_{\theta \in \Theta} \sum_{n=1}^N L(f_\theta(\mathbf{x}_n), y_n)$

□ Testing time

- Apply $f_\theta(\mathbf{x}) \rightarrow y$

The only decision(s) to make

Are we making any decisions here?

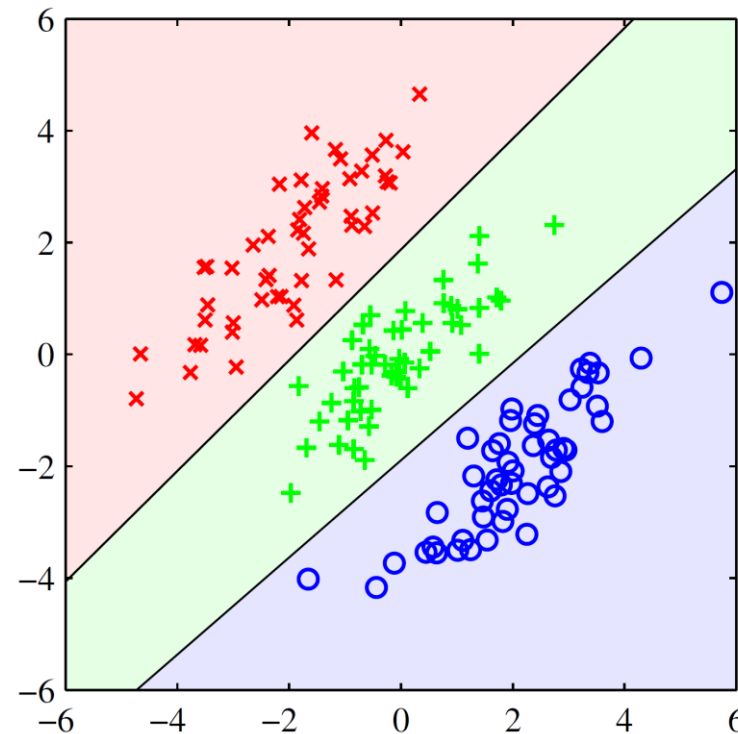


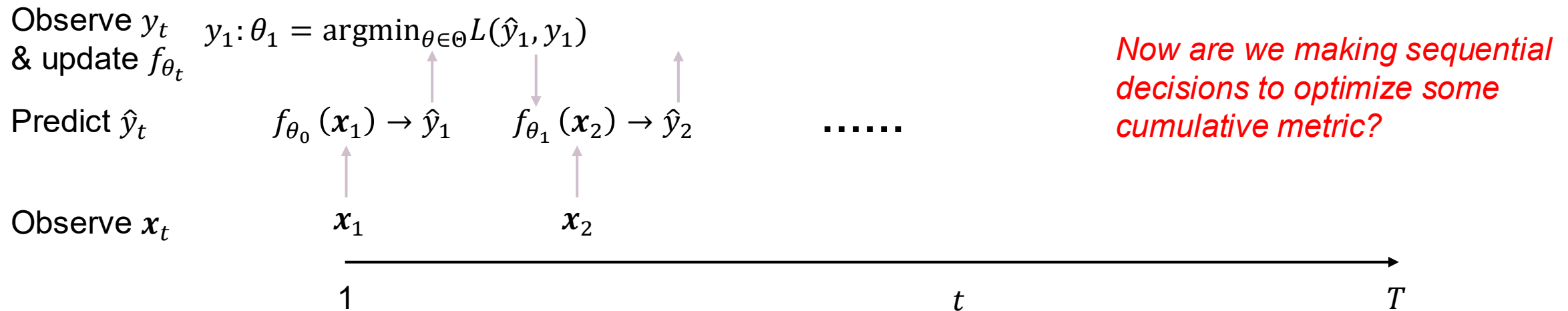
Image credit: Bishop, "Pattern Recognition and Machine Learning"

Vs. Supervised machine learning

➤ Online classification as an example

□ The hypothesis will be immediately tested

- Input: $\{(\mathbf{x}_t, y_t)\}_{t=1}^T$, where $\mathbf{x}_t \in R^d$ and $y_t \in [C]$ arrive sequentially
- Output: hypothesis $f_{\theta_t}(\mathbf{x}) \rightarrow y$ and $\hat{y}$
- Goal: $\sum_{t=1}^{T-1} \min_{\theta_t \in \Theta} L(f_{\theta_t}(\mathbf{x}_{t+1}), y_{n+1})$ ← Cumulated over T





Vs. Semi-supervised machine learning

➤ Active learning for classification as an example

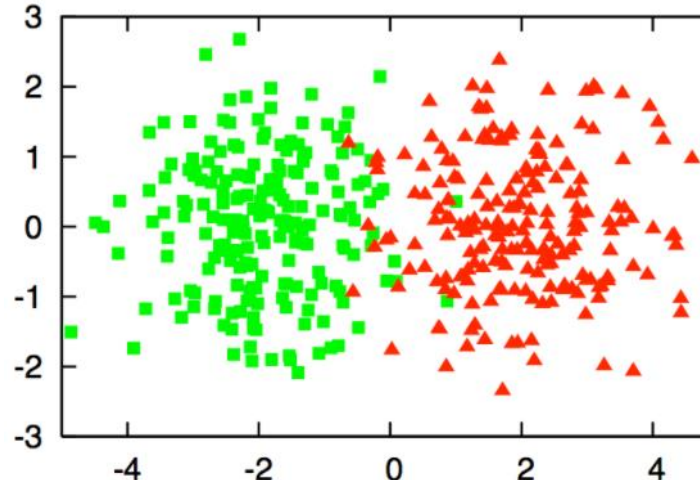
□ Training time

- Input: $\{\mathbf{x}_n\}_{n=1}^N$, where $\mathbf{x} \in R^d$
- Procedure: choose a subset of instances of size T to obtain their labels for model training
- Output: hypothesis $f_{\theta_T}(\mathbf{x}) \rightarrow y$
- Goal: $\min E_{P(\mathbf{x}, y)} [L(f_{\theta_T}(\mathbf{x}), y)]$

□ Testing time

- Apply $f_{\theta_T}(\mathbf{x}) \rightarrow y$

Budget: T queries



Now our decisions affect our observations, right!?

Full v.s., partial information

➤ Full information – in most supervised ML

- $(\mathbf{x}_t, y_t)$ is always given

➤ Partial information - bandit feedback

- Only $L(f_\theta(\mathbf{x}_t), y_t)$ is provided



$(\mathbf{x}_t, \hat{y}_t = \text{Begr})$ 

Exploitation v.s., exploration

➤ Unknown unknowns

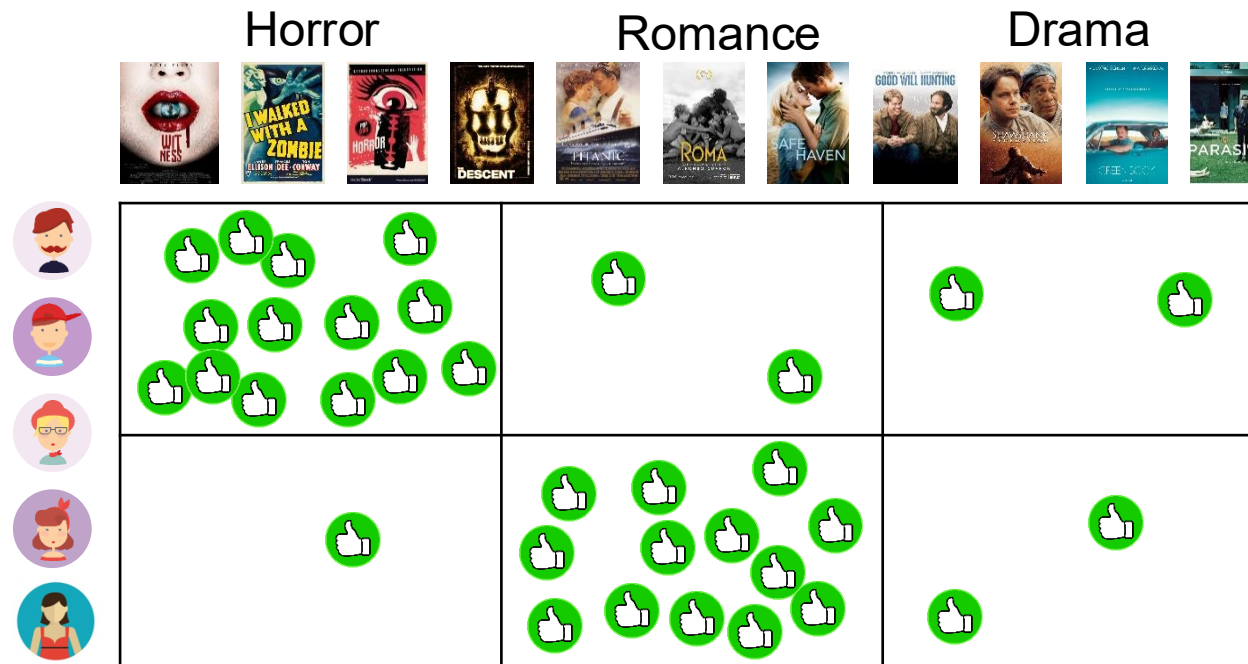


Figure credit: Schnabel et al. 2016
[SSSCJ16]

*Matthew effect: we still don't know
what we don't know!*



A quick summary

➤ Reinforcement learning

- ❑ Reward
- ❑ Partial information
- ❑ Delayed consequence
- ❑ Agent's choice affects the subsequent data it receives, i.e., non-i.i.d.

➤ Supervised learning

- ❑ Ground-truth labels
- ❑ Full information
- ❑ Immediate consequence
- ❑ Pre-determined data distribution, i.e., i.i.d.

➤ Flat structure clustering as an example

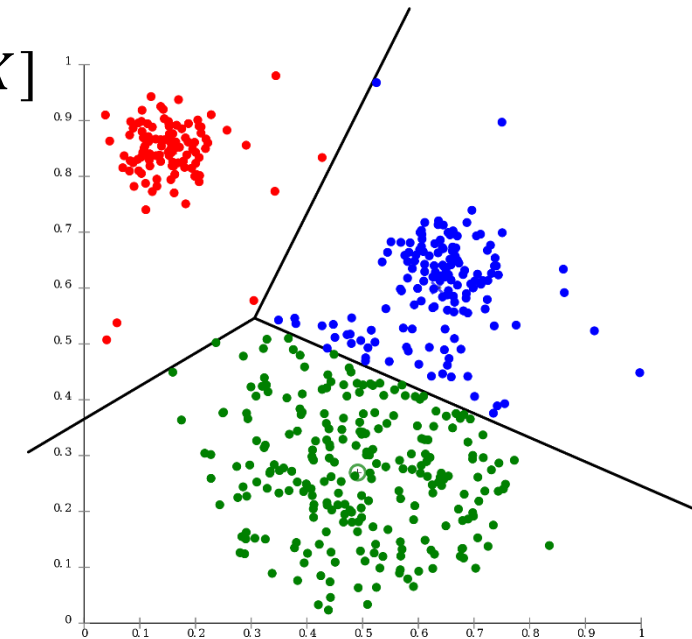
□ Training time

- Input: $\{x_n\}_{n=1}^N$, where $x \in R^d$
- Output: hypothesis $g_\theta(x) \rightarrow k$, where $k \in [K]$
- Goal: $\min \sum_{k=1}^K \sum_{g(x_m)=g(x_n)=k} L(x_m, x_n)$

□ Testing time

- Apply $g_\theta(x) \rightarrow k$

Your clustering criterion

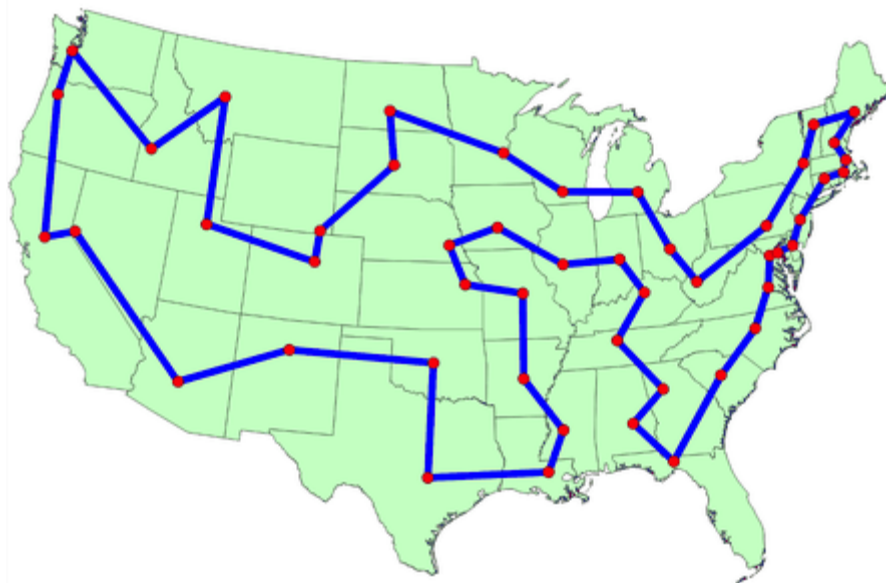


Source: https://en.wikipedia.org/wiki/Cluster_analysis

Vs. combinatorial optimization

➤ Known environment model

□ A planning problem



Traveling salesman problem

However, many combinatorial optimization problems are computationally hard. For large instances, exact algorithms may be too slow. So sometimes we use RL to learn an approximate solution:

RL view: construct a tour sequentially

- **State** s_t : current city + visited cities
- **Action** a_t : choose next unvisited city
- **Reward** $r_t = -d(i, j)$: negative travel distance
- **Return**: total negative tour length

Use a deep neural network to approximate the action-value function:

$$Q_{\theta}(s_t, a_t) \approx \mathbb{E} \left[\sum_{k=t}^T r_k \right]$$

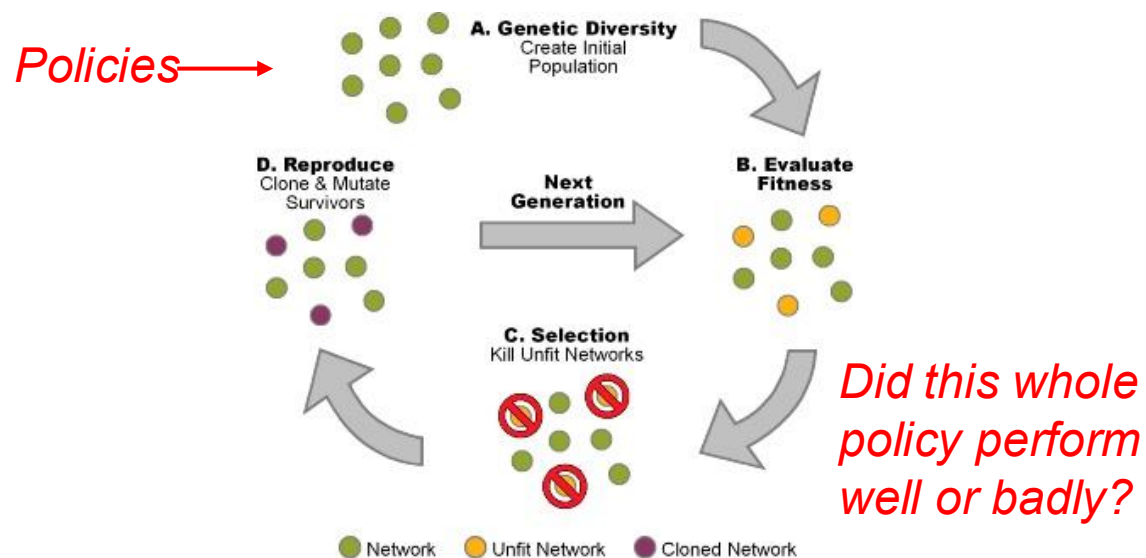
It estimates how good it is to choose city a_t from partial tour s_t .

Select the next city by: $a_t = \underset{a}{\operatorname{argmax}} Q_{\theta}(s_t, a)$

***Key idea:** use deep neural networks to learn a fast heuristic for finding short tours.*

➤ A family of trial-and-error search solutions

- ❑ Evolutionary methods search over whole policies
- ❑ Fail to leverage detailed problem structure



Genetic programming

RL algorithms often use more detailed information:

- Which state was visited?
- Which action was taken?
- What reward followed and what future state occurred?
- Which part of the policy caused the improvement or failure?

Why reinforcement learning

➤ Sequential decision making is everywhere



2016



1997

Why reinforcement learning

➤ Sequential decision making is challenging

□ Huge search space

Board size $n \times n$	3^{n^2}	Percent legal	L (legal positions) (A094777) ^[11]
1×1	3	33.33%	1
2×2	81	70.37%	57
3×3	19,683	64.40%	12,675
4×4	43,046,721	56.49%	24,318,165
5×5	847,288,609,443	48.90%	414,295,148,741
9×9	$4.43426488243 \times 10^{38}$	23.44%	$1.03919148791 \times 10^{38}$
13×13	$4.30023359390 \times 10^{80}$	8.66%	$3.72497923077 \times 10^{79}$
19×19	$1.74089650659 \times 10^{172}$	1.20%	$2.08168199382 \times 10^{170}$

Source: https://en.wikipedia.org/wiki/Go_and_mathematics



Complexity: 10^{50}

Why reinforcement learning

➤ Sequential decision making is challenging

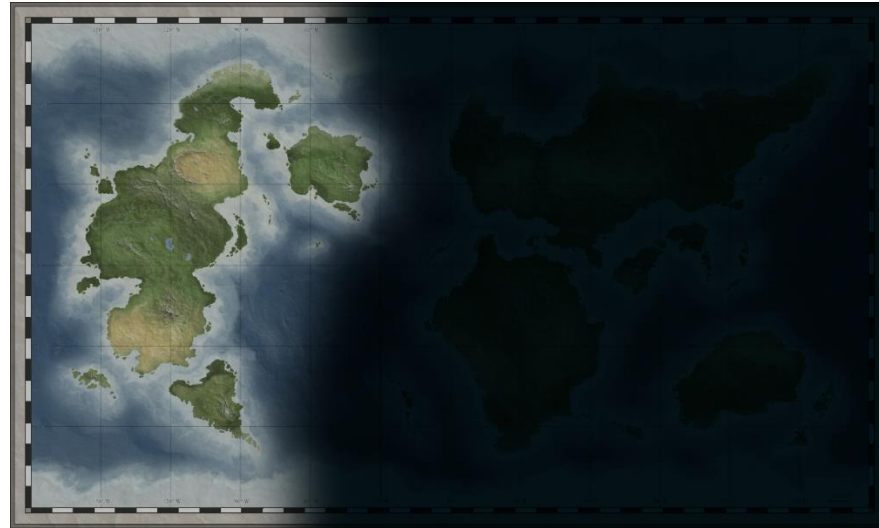
□ Huge unknown search space

	Horror				Romance				Drama			
												
	?	?	?	?		?	?	?	?	?	?	
												
												
												
												

Why reinforcement learning

➤ Sequential decision making is challenging

- Huge unknown search space
 - Supervised ML: generalize to unseen
 - RL: what to generalize



Why reinforcement learning now

➤ Computational model

- ❑ Deep learning enables sophisticated functional approximation

➤ Computational power

The system de
brute force co
parallel, RS/60
30 nodes, with
P2SC micropro
purpose VLSI

Source: <https://en.v>



on and strength^[61]

Il rating ↕ D

3,144^[51] O

ely 3,730^[51] Ma

n with 3,558^[51] Ma

Hz 5,185^[51] O

cial

5,185^[51] O

computer)

computer)

computer)

computer)

computer)

computer)

computer)

computer)

computer)



AlphaZero (20 block)

4 TPUs, single machine

[62]

Dec 2017

60:40 against AlphaGo Zero (20 block)

Source: <https://en.wikipedia.org/wiki/AlphaGo>

Why reinforcement learning now

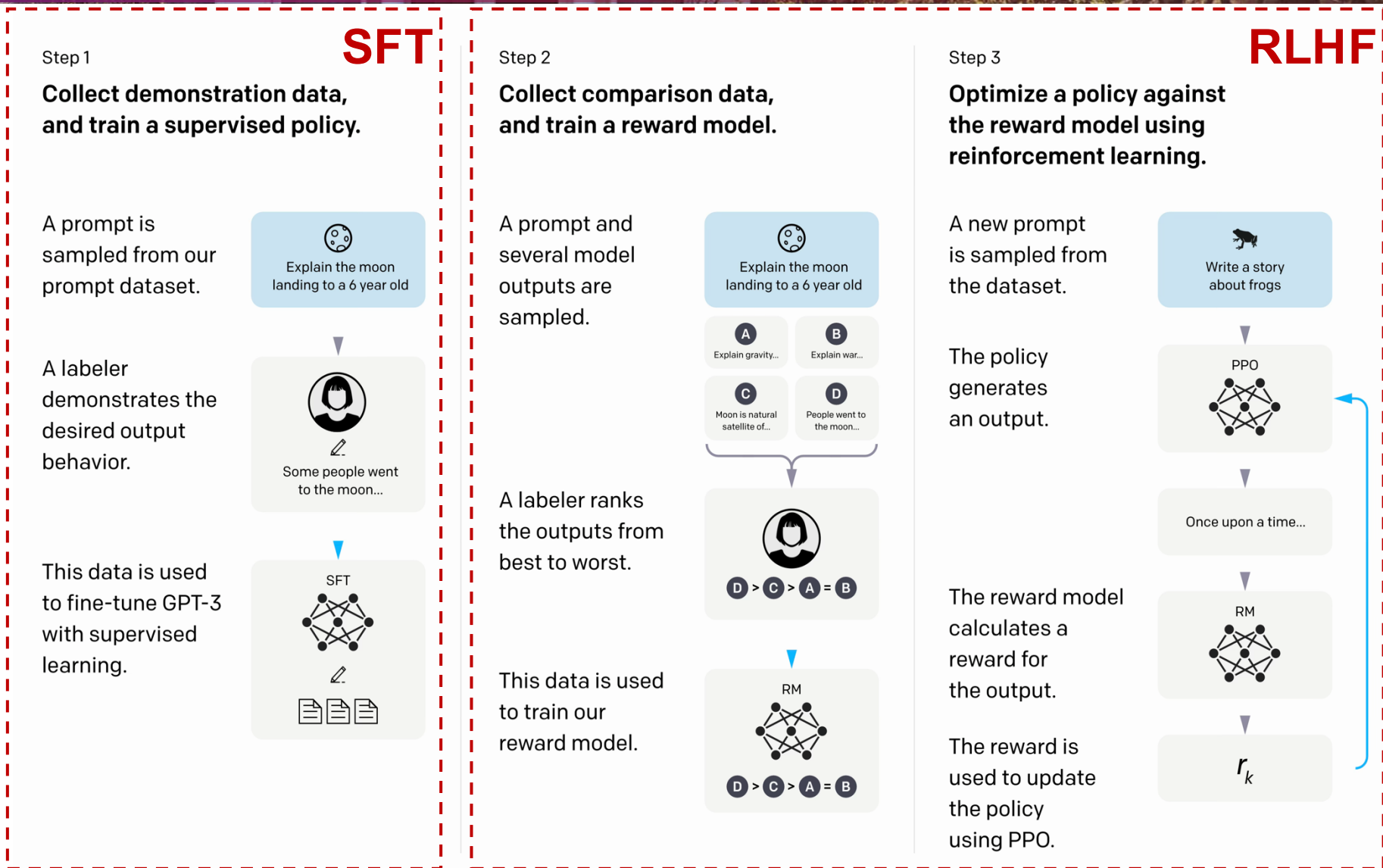
Paradigms for training Large Language Models (LLM)

Pretraining and SFT:

- Cross-entropy loss on each token
- Dense supervision
- Easy training

RL

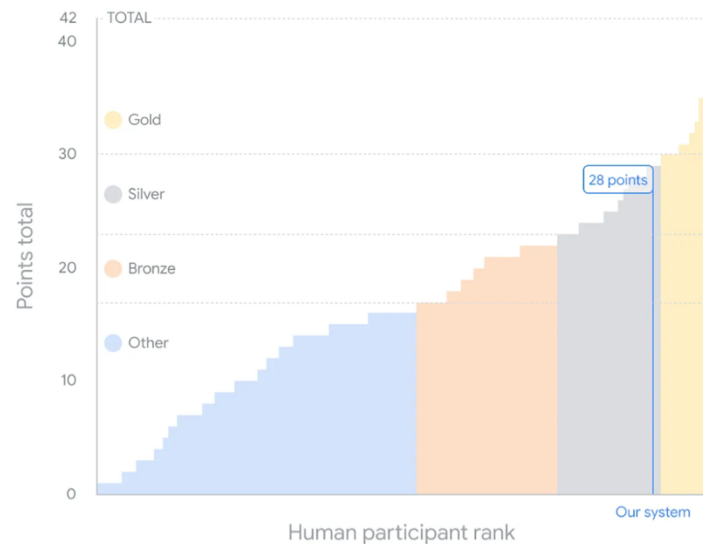
- Reward/Penalty on full response
- Sparse supervision
- Difficult training



GPT-3.5

Why reinforcement learning now

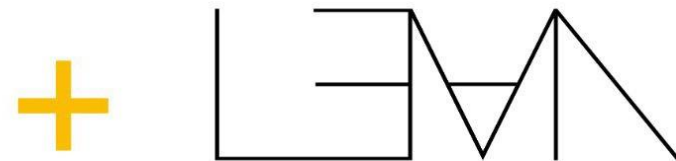
Score on IMO 2024 problems



AlphaZero is the agent.



Lean is the environment.



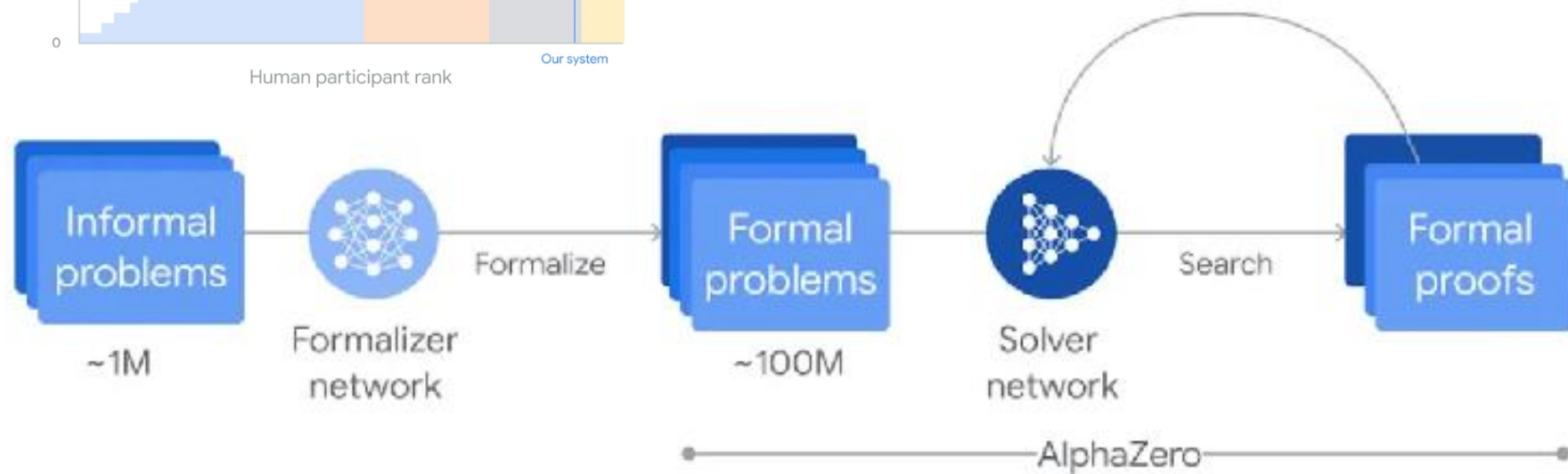
Paradigms for training Large Language Models (LLM)

Pretraining and SFT:

- Cross-entropy loss on each token
- Dense supervision
- Easy training

RL

- Reward/Penalty on full response
- Sparse supervision
- Difficult training

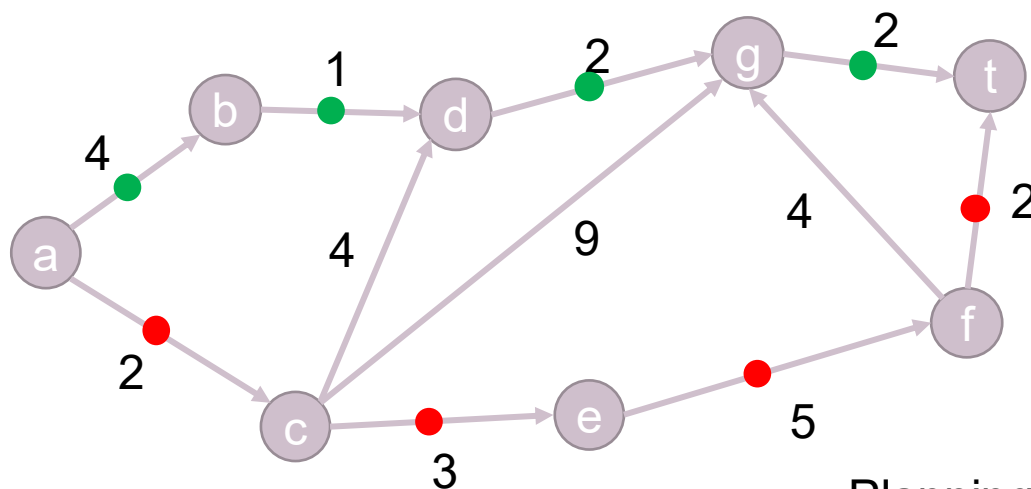


AlphaProof

How to do reinforcement learning

➤ With a known environment

□ Planning - a classical AI search problem



Dijkstra's algorithm

Maintain the best cost discovered so far from the starting node a to each node. Repeatedly expand the unprocessed node with the **smallest accumulated cost from a** , updating its neighbors:

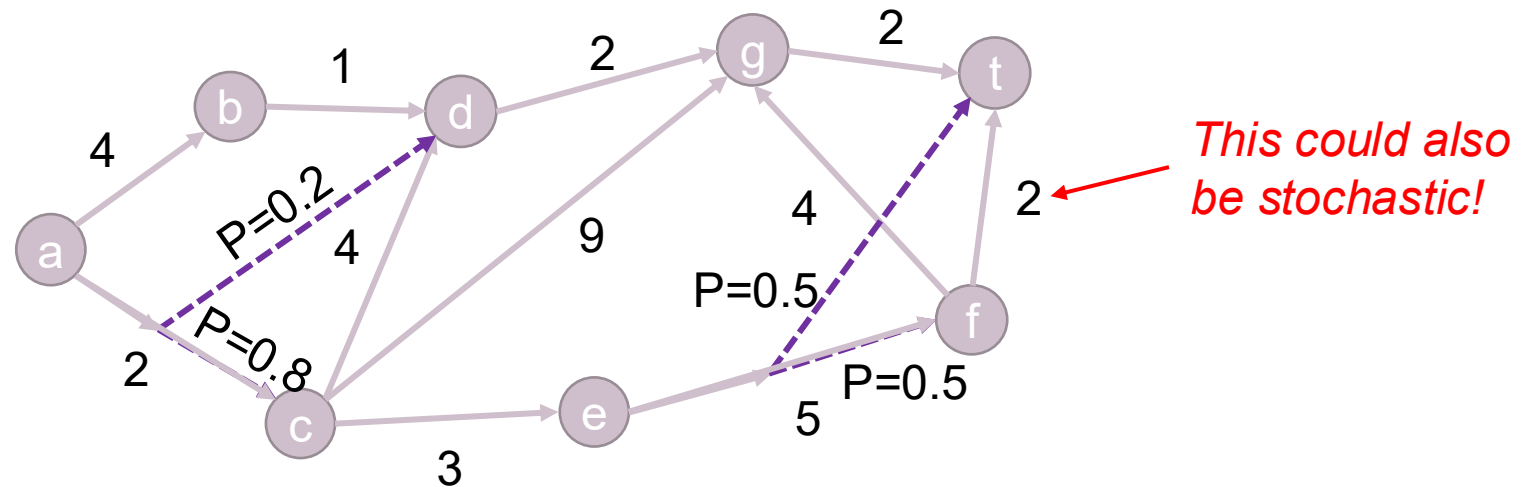
$$D(v) \leftarrow \min\{D(v), D(u) + w(u, v)\}.$$

Planning for the long-term is necessary

How to do reinforcement learning

➤ With a known but stochastic environment

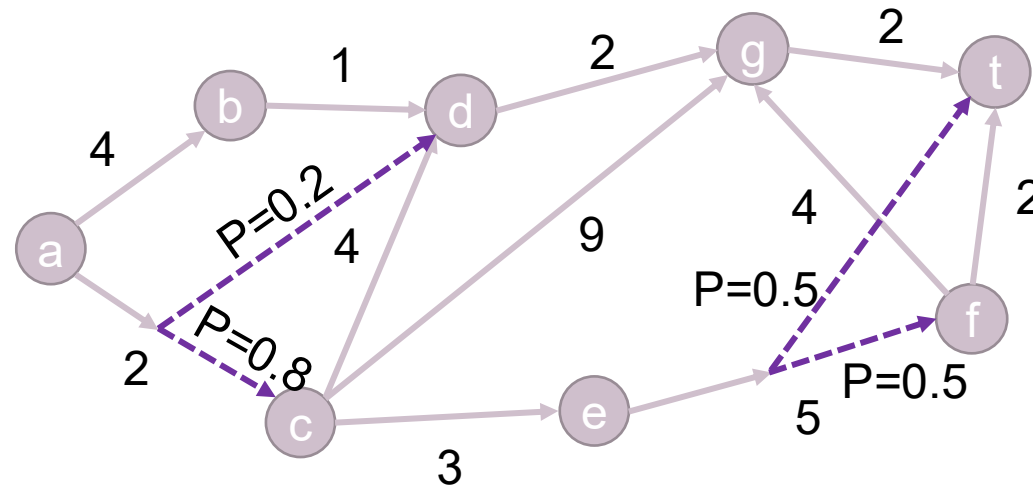
□ Planning – value/policy estimation



How to do reinforcement learning

➤ With an unknown and stochastic environment

□ Planning



How to do reinforcement learning

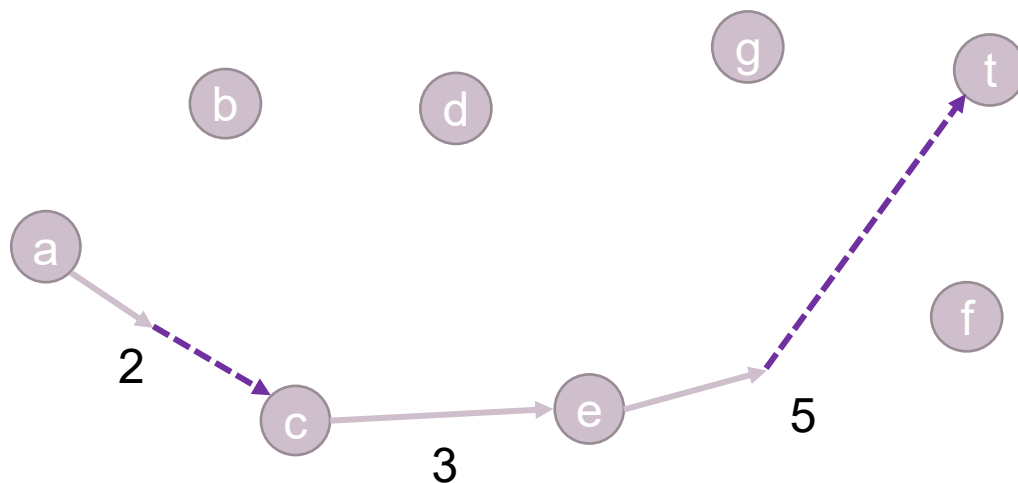
➤ With an unknown and stochastic environment

□ Planning while learning

• Trial

Experience:

$a \rightarrow c \rightarrow e \rightarrow t$



How to do reinforcement learning

➤ With an unknown and stochastic environment

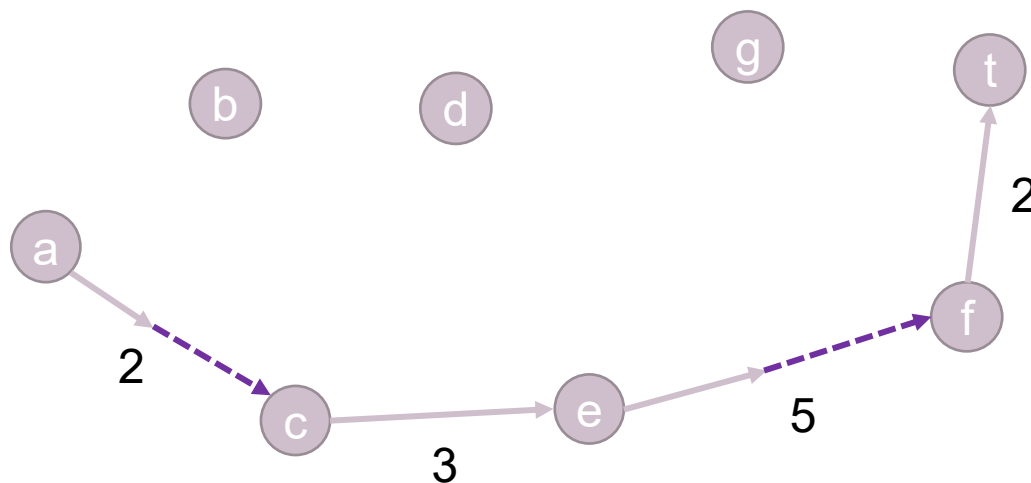
□ Planning while learning

- Trial and error

Experience:

$a \rightarrow c \rightarrow e \rightarrow t$

$a \rightarrow c \rightarrow e \rightarrow f \rightarrow t$



How to do reinforcement learning

➤ With an unknown and stochastic environment

□ Planning while learning

- Trial and error

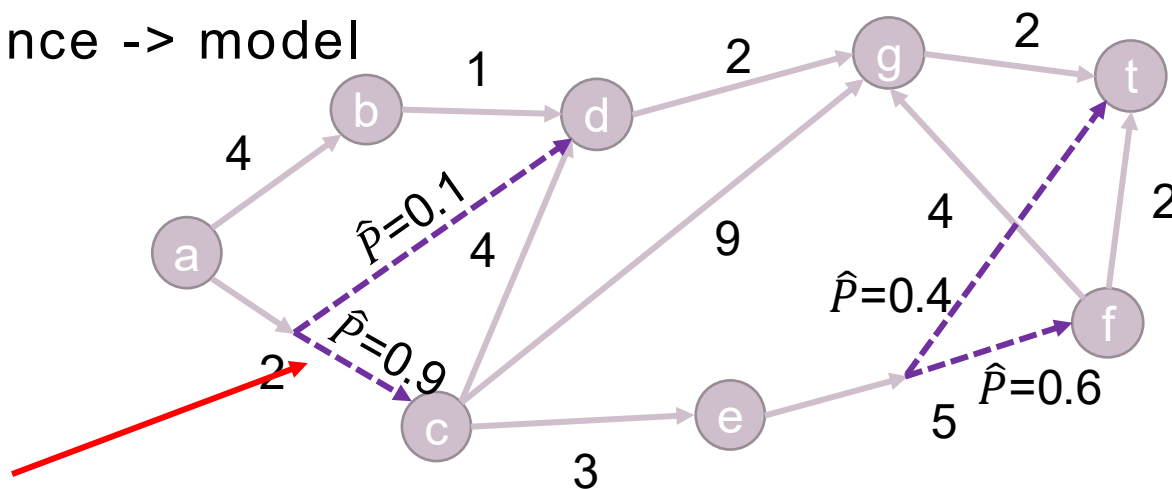
□ Model-based

- Experience -> model

Experience:

$a \rightarrow c \rightarrow e \rightarrow t$

$a \rightarrow c \rightarrow e \rightarrow f \rightarrow t$



Convergence of the model matters!

How to do reinforcement learning

➤ With an unknown and stochastic environment

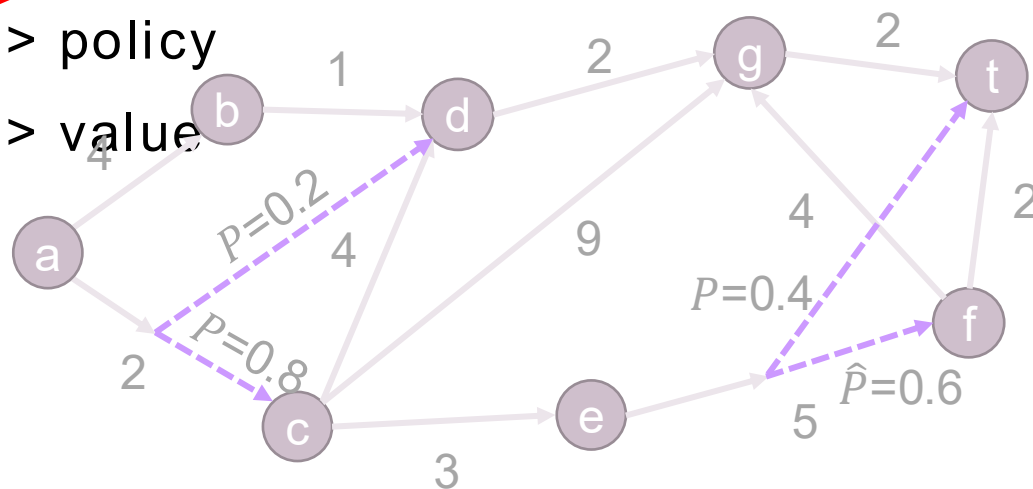
□ Planning while learning

- Trial and error

□ Model-free

- Experience $\rightarrow$ policy
- Experience $\rightarrow$ value

How do we get such experiences matters!



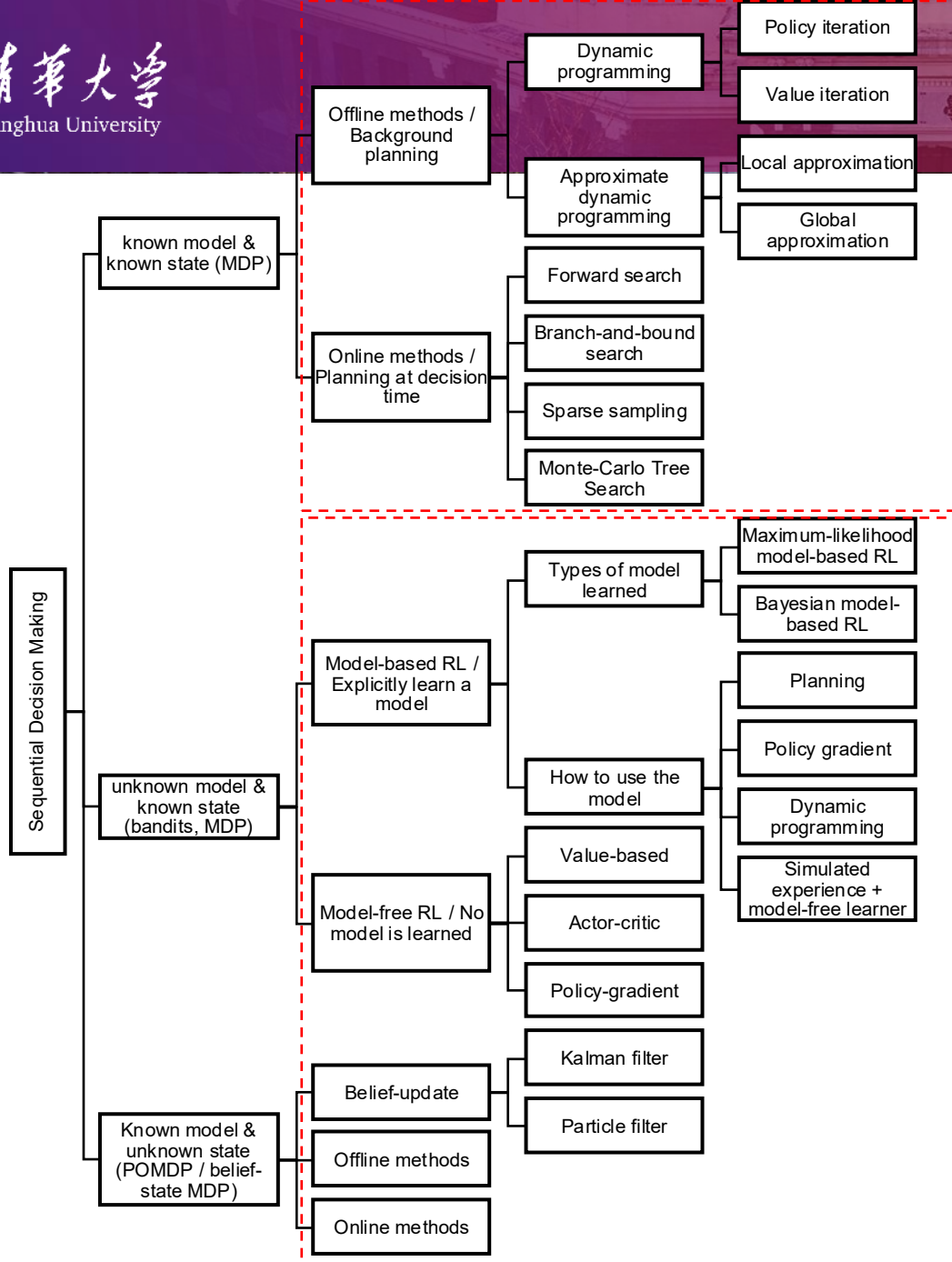
Experience:

$a \rightarrow c \rightarrow e \rightarrow t$

$a \rightarrow c \rightarrow e \rightarrow f \rightarrow t$

$\pi(a)$	Up	$P=0.3$
	Down	$P=0.7$
$v(a)$	9	Up
	7.6	Down

A very coarse taxonomy of solutions



→ planning

And of course, there are also many extended problems:

- MDP / POMDP with constraints / budgets
- Risk-aware MDP / POMDP
- Hierarchical MDP / POMDP
- and various multi-agent settings (closely related to algorithmic game theory, possibly an even bigger tree than what we see here)
- ...

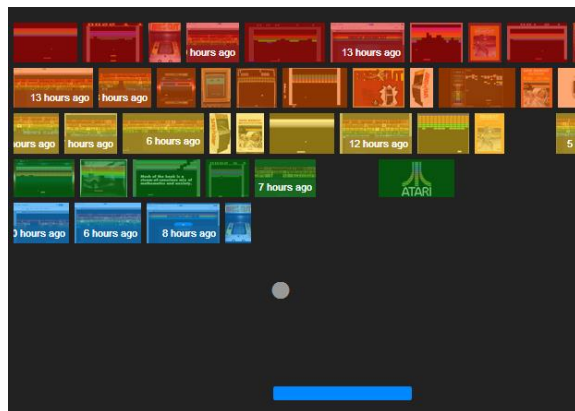
→ reinforcement learning

Places where neural networks can be used:

- Approximate $V(x)$ or $Q(x, a)$ function
- Approximate policy $\pi(x)$
- Approximate dynamics model (either sample model or distribution model)

RL in practice is challenging

- ❑ Learning by trial and error is expensive
 - We need an environment to repeatedly interact with



Environment defined by

game rules



physics

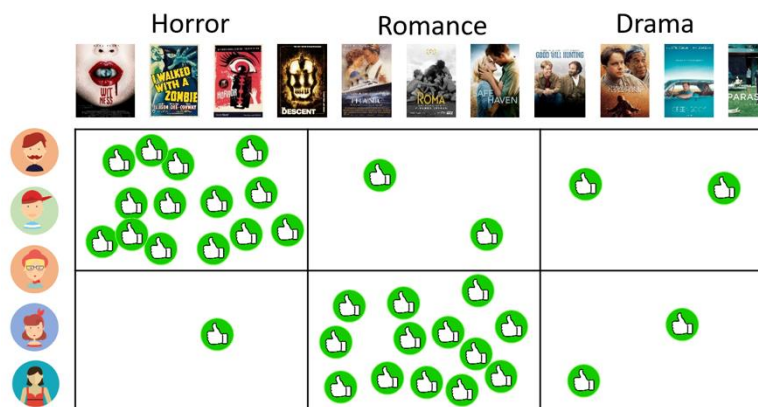


events and processes

Simulation is easy

RL in practice is challenging

- ❑ Learning by trial and error is expensive
 - We need an environment to repeatedly interact with



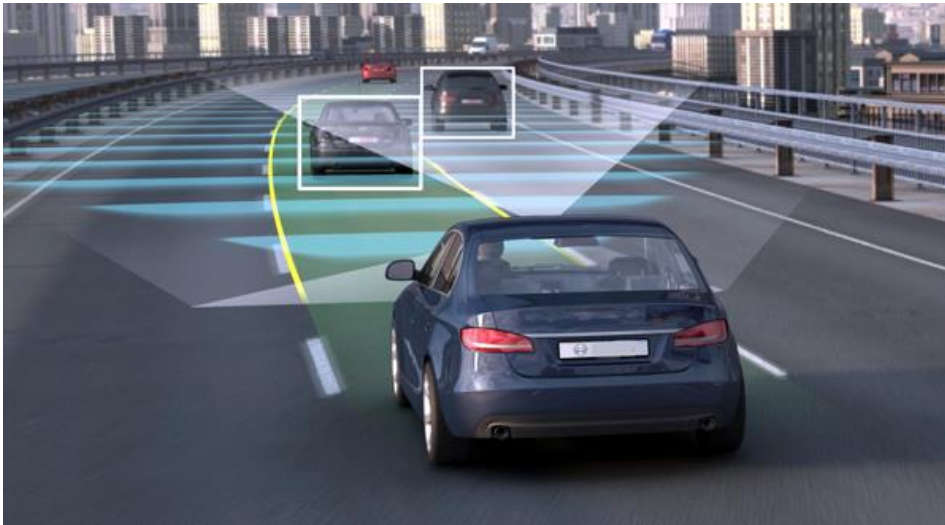
Environment defined by

Users' behaviors

Simulation is hard!

RL in practice is challenging

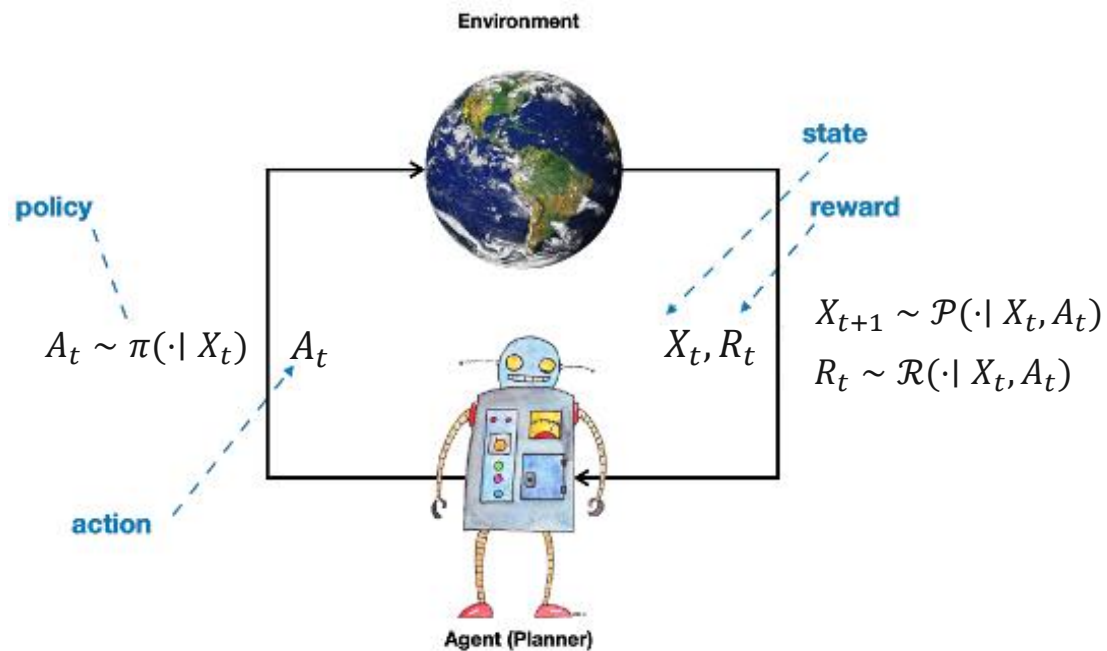
- ❑ There are also safety, privacy, and ethic concerns in exploration
 - We are dealing with unknown unknowns
 - We are learning from explorations



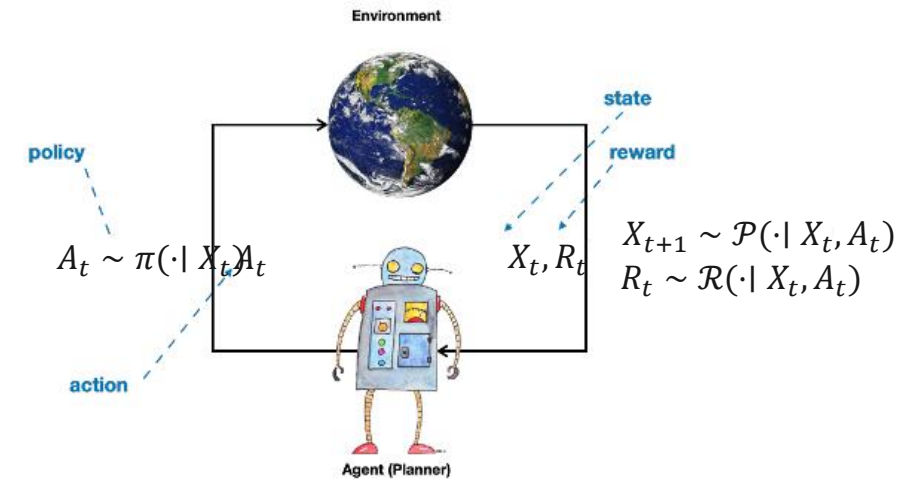
Terminologies



In RL, we often talk about an agent and its environment, and their interaction.

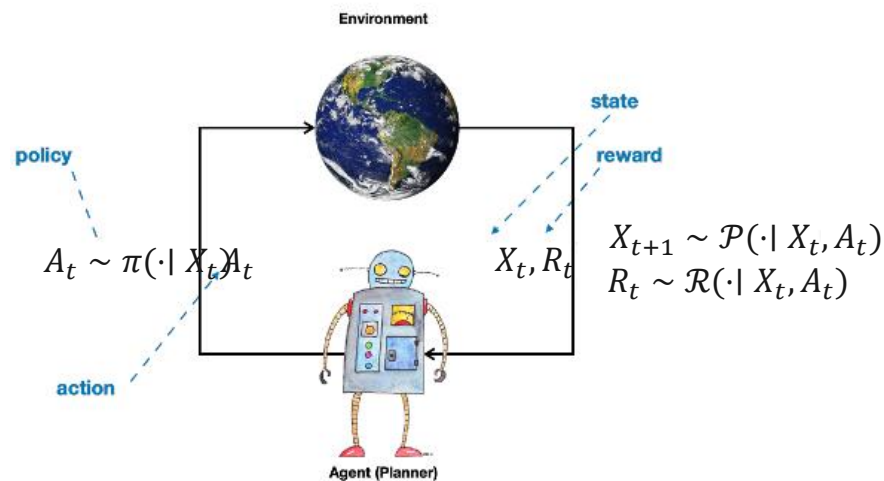


- Agent: decision maker and/or learner
 - robot
 - medical diagnosis and treatment system
 - air conditioning system
- Environment: anything outside the agent with which it interacts and attempts to control.
 - physical world outside the robot
 - patient's body
 - room



At time $t = 1, 2, \dots$, the interaction of the agent and the environment is as follows:

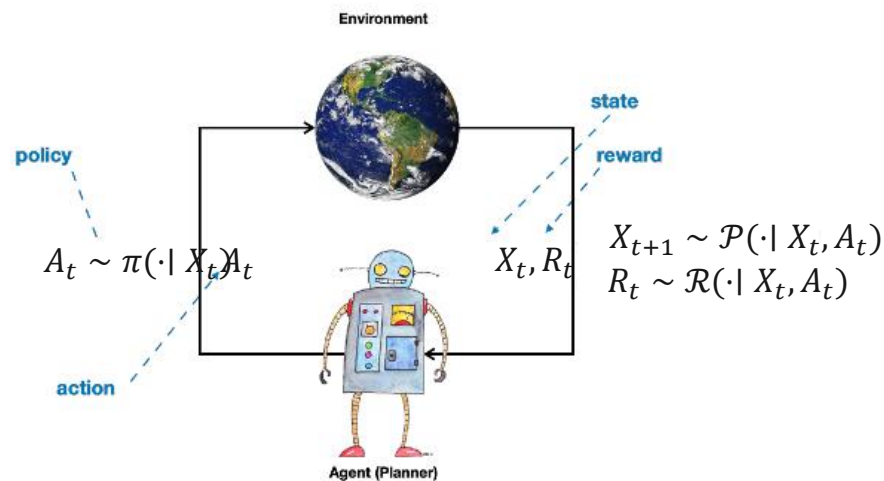
- the agent observes its state X_t in the environment.
 - Examples: position of the robot, vital information of a patient, the room temperature, etc.
- The agent picks an action A_t according to its policy π , e.g., $A_t = \pi(X_t)$ or $A_t \sim \pi(\cdot | X_t)$.
- The state of the agent in the environment changes and becomes X_{t+1} according to transition probability kernel (or distribution), i.e., $X_{t+1} \sim \mathcal{P}(\cdot | X_t, A_t)$.
- The agent also receives a reward signal R_t , i.e., $R_t \sim \mathcal{R}(\cdot | X_t, A_t)$.



State: A variable that summarizes whatever has happened to the agent so far.

Policy: Action selection mechanism. Usually a mapping from states to actions. It can be deterministic ($A_t = \pi(X_t)$) or stochastic ($A_t \sim \pi(\cdot | X_t)$).

Transition probability kernel: Describes the dynamics. For example, a set of electromechanical equations describing how the position of the robot (including its joints) change when a certain command is sent to its motor. Or how the patient's physiology changes after the administration of the treatment.

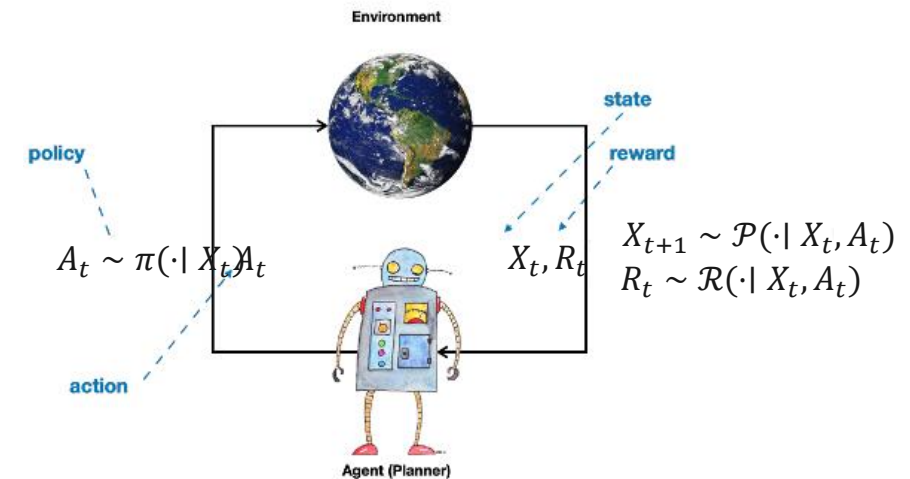


Reward: A real number specifying the immediate desirability of the choice of action A_t at the state X_t (possibly leading to state X_{t+1}) has been. Examples:

- Positive if robot successfully picks up an object, negative if it breaks the object.
- Infection subsides
- The room temperature becomes comfortable.

Remark

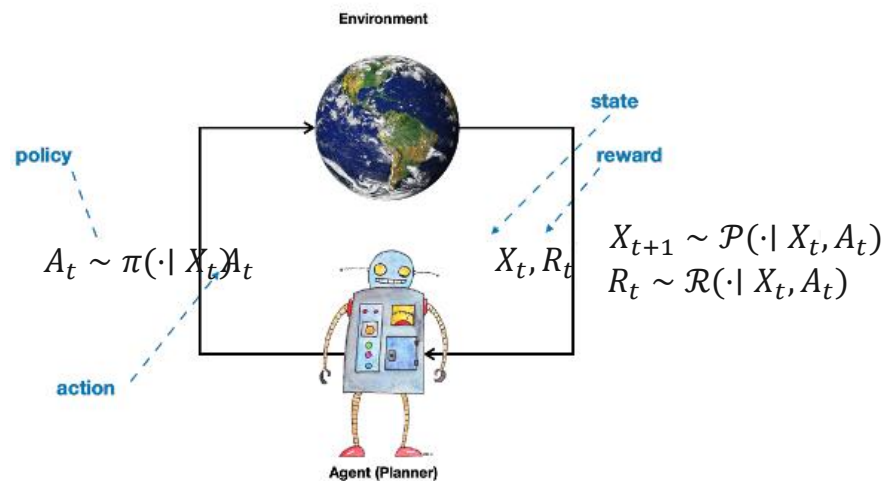
The reward signal/function/distribution only encodes the desirability of the action from the immediate perspective. A good action now may not be good in the long-term.



This process repeats and as a result, the agent receives a sequence of state, actions, and rewards:

$$X_1, A_1, R_1, X_2, A_2, R_2, \dots$$

This sequence might terminate after a fixed number of time steps (say, T), or until the agent gets to a certain region of the state space, or it might continue forever.



Let us formally define some important concepts that we require throughout the course.
Beforehand, some commonly used notations:

Given a space Ω .

- $\mathcal{M}(\Omega)$: the space of all probability distributions defined over the space Ω .
- $B(\Omega)$: the space of all bounded functions defined over Ω

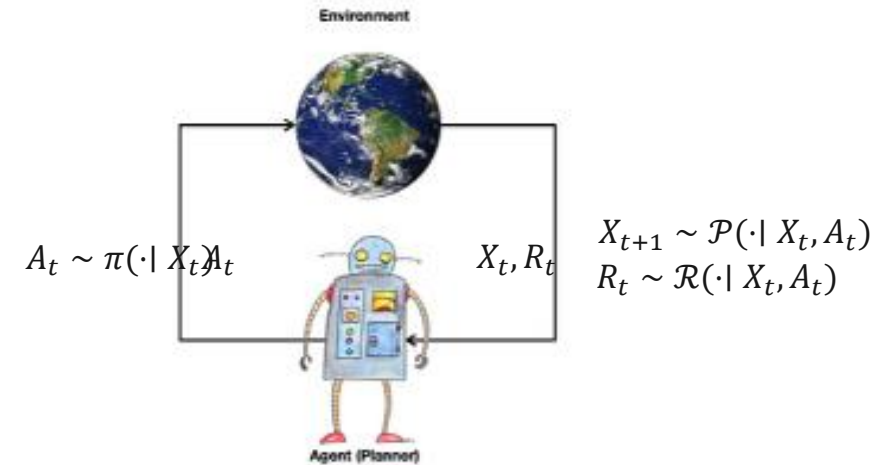
Examples: $\Omega = \{1, 2, \dots, n\}, \mathbb{N}, \mathbb{R}, \mathbb{R}^d$, etc.

$\mathcal{M}(\mathbb{R})$: The space of distributions on the real line

$B(\mathbb{R})$: The space of bounded functions on the real line

Definition

A discounted MDP is a 5-tuple $(\mathcal{X}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where $\mathcal{X}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{P}: \mathcal{X} \times \mathcal{A} \rightarrow \mathcal{M}(\mathcal{X})$ is the transition probability kernel with domain $\mathcal{X} \times \mathcal{A}$, $\mathcal{R}: \mathcal{X} \times \mathcal{A} \rightarrow \mathcal{M}(\mathbb{R})$ is the immediate reward distribution, and $0 \leq \gamma < 1$ is the discount factor.



MDPs encode the temporal evolution of a discrete-time stochastic process controlled by an agent.

- Initial state $X_1 \sim \rho$ with $\rho \in \mathcal{M}(\mathcal{X})$.
- Agent chooses action $A_t \in \mathcal{A}$.
- Agent goes to $X_{t+1} \sim \mathcal{P}(\cdot | X_t, A_t)$ and receives reward $R_t \sim \mathcal{R}(\cdot | X_t, A_t)$.
- The process repeats. The trajectory (or rollout) is $\xi = (X_1, A_1, R_1, X_2, A_2, R_2, \dots)$, which is random.

Remark

The reward distribution could also depend on the next-state X_{t+1} . In that case, we would have a different reward kernel $\mathcal{R}'$ and the reward would be $R_t \sim \mathcal{R}'(\cdot | X_t, A_t, X_{t+1})$. But we can absorb the dynamics within $\mathcal{R}$, i.e.,

$$\mathcal{R}(\cdot | x, a) = \int \mathcal{R}'(\cdot | x, a, x') \mathcal{P}(dx' | x, a).$$

This is a general framework.

State space:

- Finite: $\mathcal{X} = \{x_1, x_2, \dots, x_n\}$ (or $\mathcal{X} = \{1, 2, \dots, n\}$) with $n < \infty$
- Infinite but countable: $\mathcal{X} = \{x_1, x_2, \dots\}$ (or $\mathcal{X} = \mathbb{N}$)
- Continuous: $\mathcal{X} \subset \mathbb{R}^d$

Dynamics:

- Stochastic
- Deterministic

- When $\mathcal{X} = \{x_1, x_2, \dots, x_m\}$, the transition probability kernel $\mathcal{P}(\cdot|\cdot, a)$ is a matrix for any $a \in \mathcal{A}$. For example,

$$\mathcal{P}(\cdot|\cdot, a_1) = \begin{bmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \end{bmatrix}, \quad \mathcal{P}(\cdot|\cdot, a_2) = \begin{bmatrix} 0.8 & 0.2 \\ 0.1 & 0.9 \end{bmatrix}.$$

- a dynamical system described by $x_{t+1} = f(x_t, a_t)$ where $x \in \mathbb{R}^m$, $a \in \mathbb{R}^n$, and $f: \mathbb{R}^m \times \mathbb{R}^n \rightarrow \mathbb{R}^m$. For example,

$$f(x, a) = cx + a, \quad f(x, a) = ax(1 - x)$$

Remark

A deterministic dynamical system always behave exactly the same given the same starting state and action. They can be described by the transition function $f: \mathcal{X} \times \mathcal{A} \rightarrow \mathcal{X}$ and $x_{t+1} = f(x_t, a_t)$.

A policy is the action selection mechanism of the agent. It indicates what action is chosen at each time. Let us start from a simplified, yet commonly used, definition: A policy is a mapping from the current state x to an action. It can be

- deterministic: $\pi(x)$
- stochastic: $\pi(\cdot | x)$

Examples:

- When the agent selects action a_1 in state x_1 with probability 0.2 and action a_2 in the same state with probability 0.8, we have

$$\pi(a | x) = \begin{cases} 0.8 & a = a_1 \\ 0.2 & a = a_2 \end{cases}$$

- When the agent always selects action a_2 in state x , this means that it has a deterministic policy and we denote it by $\pi(x) = a_2$.

The definition of the policy is more general. Let us briefly review it.

Definition

A **policy** is a sequence $\bar{\pi} = \{\pi_1, \pi_2, \dots\}$ such that for each t ,

$$\pi_t(a_t | X_1, A_1, X_2, A_2, \dots, X_{t-1}, A_{t-1}, X_t)$$

is a stochastic kernel on $\mathcal{A}$ given $\underbrace{\mathcal{X} \times \mathcal{A} \times \dots \times \mathcal{X} \times \mathcal{A} \times \mathcal{X}}_{2t-1 \text{ elements}}$ satisfying

$$\pi_t(\mathcal{A} | X_1, A_1, X_2, A_2, \dots, X_{t-1}, A_{t-1}, X_t) = 1$$

for every $(X_1, A_1, X_2, A_2, \dots, X_{t-1}, A_{t-1}, X_t)$.

This means that the policy can depend on time t , and be a mapping from all previous observed states and selected actions.

Definition

If π_t is parametrized only by X_t , that is

$$\pi_t(\cdot | X_1, A_1, X_2, A_2, \dots, X_{t-1}, A_{t-1}, X_t) = \pi_t(\cdot | X_t),$$

$\bar{\pi}$ is a **Markov** policy.

If for each t and $(X_1, A_1, X_2, A_2, \dots, X_{t-1}, A_{t-1}, X_t)$, the policy π_t assigns mass one to a single point in $\mathcal{A}$, $\bar{\pi}$ is called a **deterministic (nonrandomized)** policy; if it assigns a distribution over $\mathcal{A}$, it is called **stochastic** or **randomized** policy.

If $\bar{\pi}$ is a Markov policy in the form of $\bar{\pi} = (\pi, \pi, \dots)$, it is called a **stationary** policy.

A policy $\pi(\cdot | x)$ is a stationary Markov policy. We often work with such policies. If it is also deterministic, we denote it by $\pi(x)$.

An agent is “following” a Markov stationary policy π whenever A_t is selected according to the policy $\pi(\cdot|X_t)$, i.e., $A_t = \pi(X_t)$ (deterministic) or $A_t \sim \pi(\cdot|X_t)$ (stochastic).

The policy π induces two transition probability kernels $\mathcal{P}^\pi : \mathcal{X} \rightarrow \mathcal{M}(\mathcal{X})$ and $\mathcal{P}^\pi : \mathcal{X} \times \mathcal{A} \rightarrow \mathcal{M}(\mathcal{X} \times \mathcal{A})$. For a subset A of $\mathcal{X}$ and a subset B of $\mathcal{X} \times \mathcal{A}$ and a deterministic policy π , denote

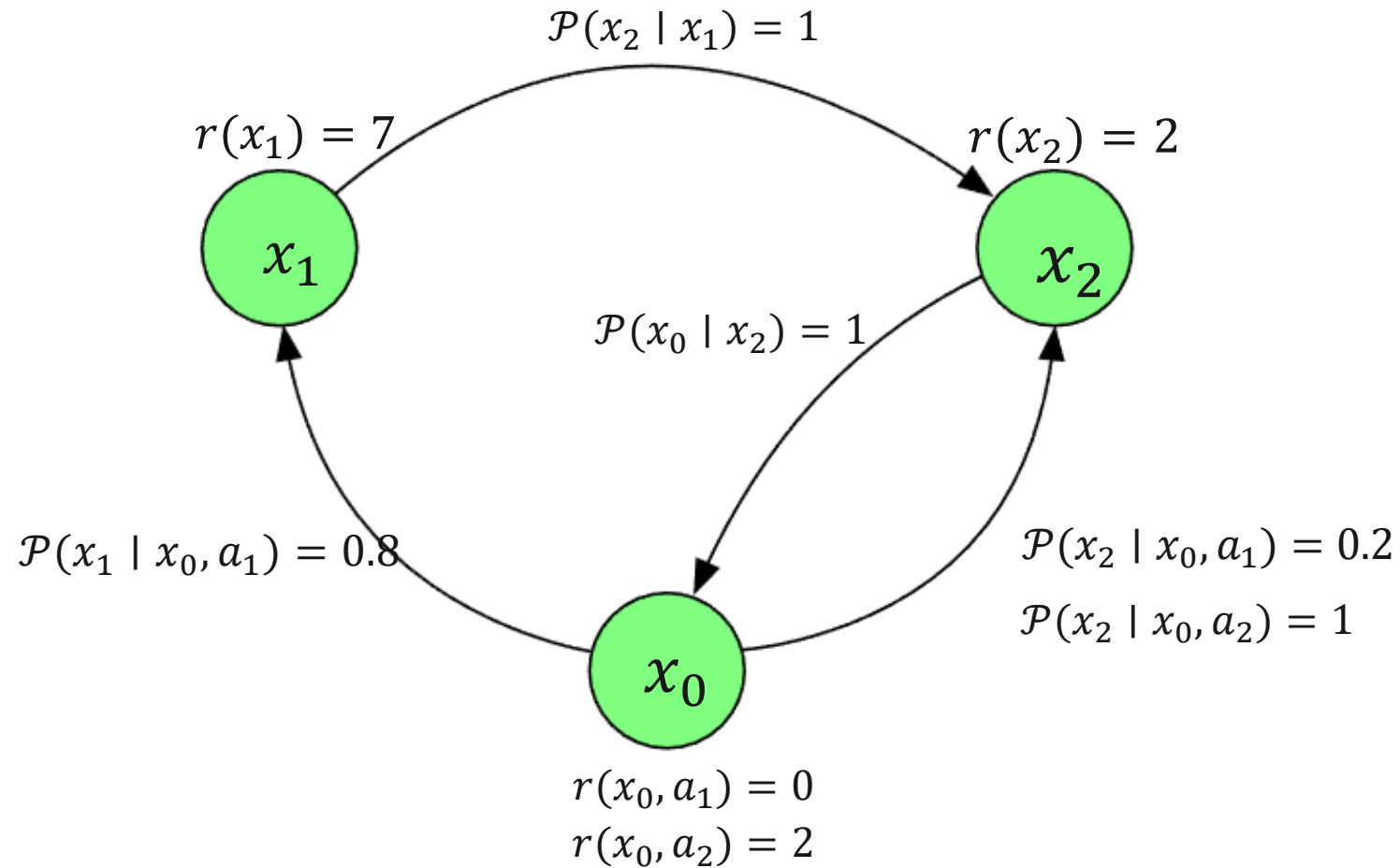
$$(\mathcal{P}^\pi)(A|x) \triangleq \int_{\mathcal{X}} \mathcal{P}(\mathrm{d}y|x, \pi(x)) \mathbb{I}_{\{y \in A\}},$$
$$(\mathcal{P}^\pi)(B|x, a) \triangleq \int_{\mathcal{X}} \mathcal{P}(\mathrm{d}y|x, a) \mathbb{I}_{\{(y, \pi(y)) \in B\}}.$$

When we have a countable state-action space, we sometimes use summation instead of integrals. For example,

$$(\mathcal{P}^\pi)(A|x) \triangleq \sum_{y \in \mathcal{X}} \mathcal{P}(y|x, \pi(x)) \mathbb{I}_{\{y \in A\}} = \sum_{y \in A} \mathcal{P}(y|x, \pi(x)).$$

So for a particular $y \in \mathcal{X}$, we have $(\mathcal{P}^\pi)(y|x) = \mathcal{P}(y|x, \pi(x))$.

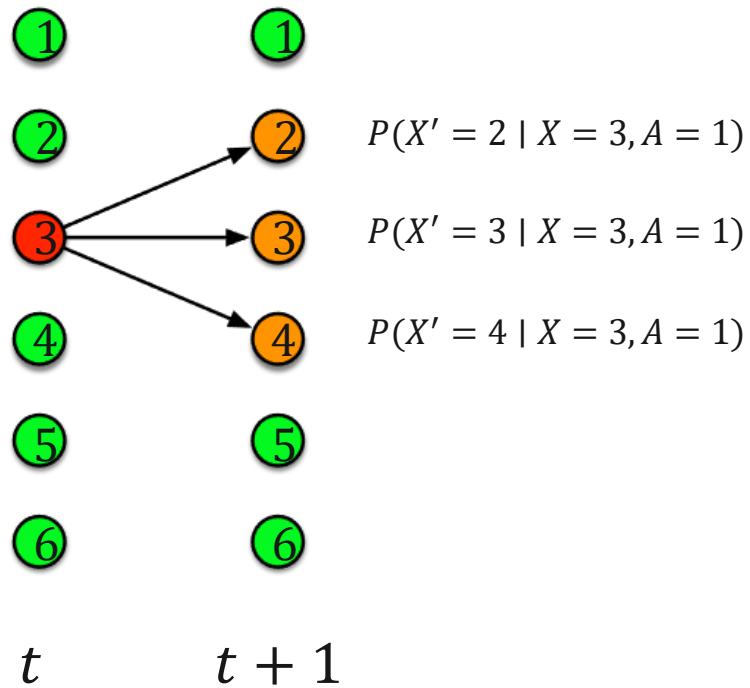
Also we can extend the definition of $\mathcal{P}^\pi$ to following a policy for m -steps ($m \geq 1$) inductively. We use $(\mathcal{P}^\pi)^m$ to denote such a transition kernel.



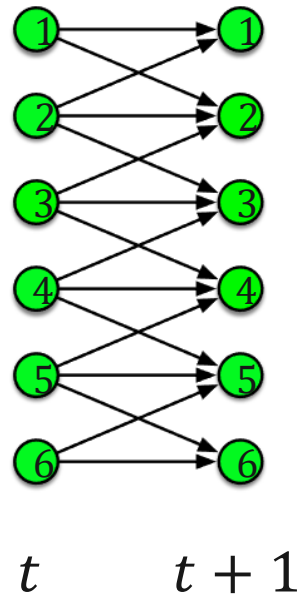
- What are the deterministic policies here? Hint: There are two!
- Write down $\mathcal{P}^{\pi_1}$ and $\mathcal{P}^{\pi_2}$.



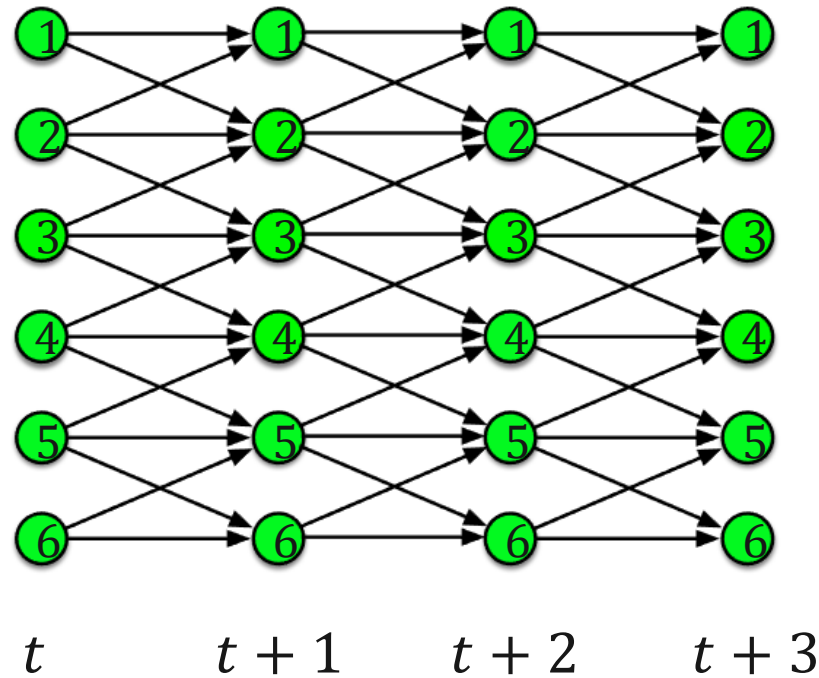
The state space $\mathcal{X}$ has 6 states. At time t , the agent is at state 3.



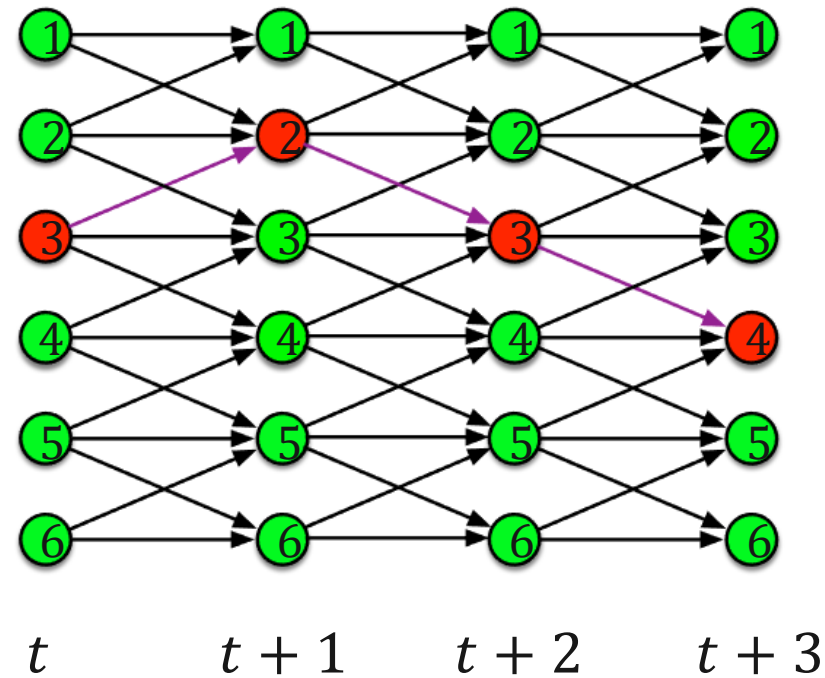
Depending on the choice of action, it can go to one of the other states at time $t + 1$.



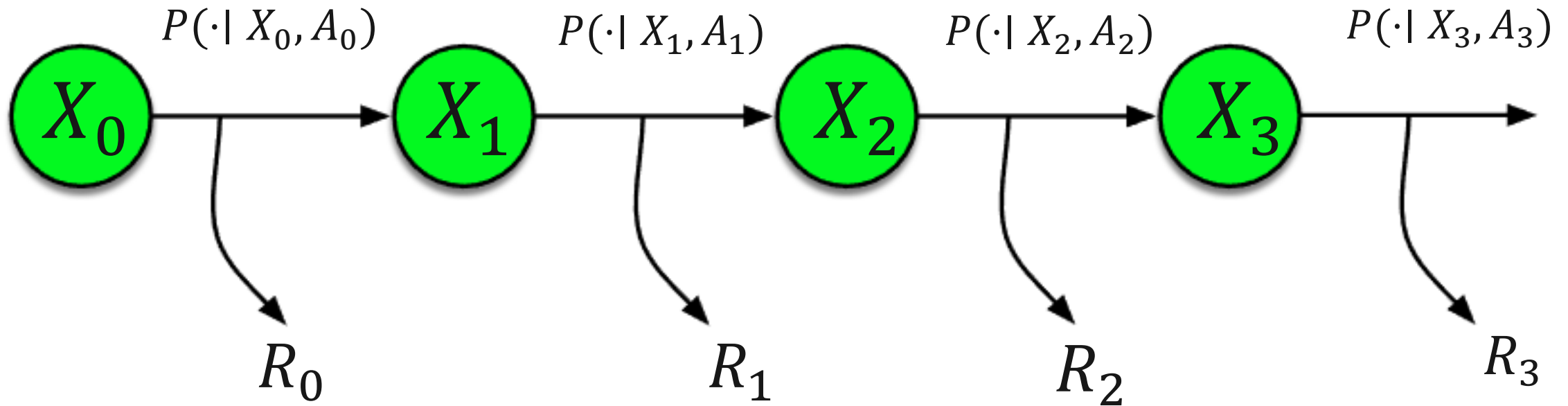
Of course, this is more general. If the agent is at state x , it will be at another state X' , depending on the choice of action a : $X' \sim \mathcal{P}(\cdot | x, a)$.



And this process extends for multiple (many or infinity) time steps.



A particular path in the state space is highlighted.



Each particular path generates a trajectory (or rollout)

$$\xi = (X_1, A_1, R_1, X_2, A_2, R_2, \dots).$$



Thank you!
Q&A